

Week 6

Understanding Neural Networks

Nebius Academy



About me

My name is Tatiana Gaintseva, I am a PhD candidate in AI research at Digital Environment Research Institute, Queen Mary University of London

My research interest lie in representation engineering and examining and utilising features of intermediate model representations

I have taught ML and DL in MSU, MIPT, YSDA, Deep Learning School and other places.



More about me here:

atmyre.github.io

Plan

- Logistic Regression -> Neural Network
- Training of neural networks
- Activation functions in neural networks
- Building neural network on PyTorch!

Logistic Regression ->
Neural Network

Logistic Regression

Consider a binary classification task

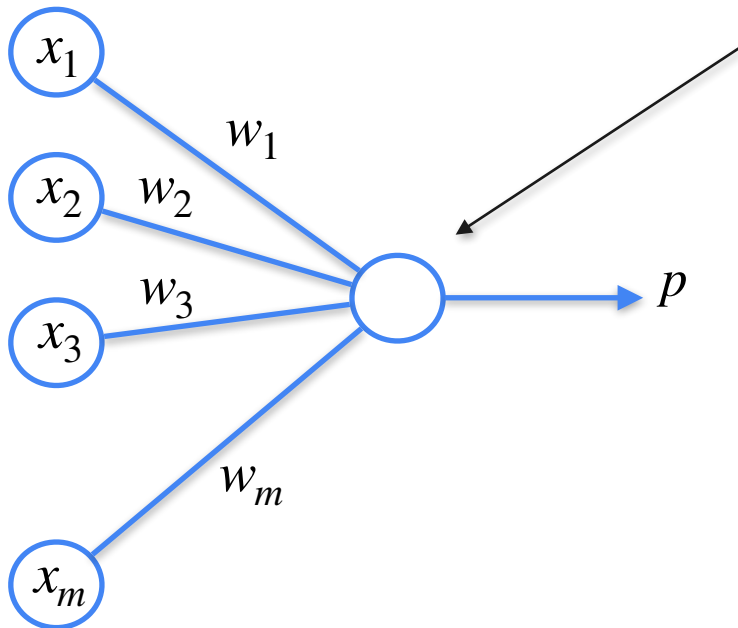
	F_1	F_2	F_3	$\dots$	F_m	y
Object 1	0.1	1.34	3.45		- 1.1	0
Object 2	1.0	2.3	- 4.5		- 0.9	1
Object 3	0.98	1.24	- 1.34		- 0.87	1
Object 4	2.31	0.56	- 0.78		1.45	0
	w_1	w_2	w_3		w_m	

Formula for Logistic Regression: $p = \sigma \left(w_o + \sum x_i w_i \right)$

Training of LR consists of finding optimal values for parameters w_i

Logistic Regression

We can illustrate LR as follows:



$$p = \sigma \left(w_o + \sum x_i w_i \right)$$

Summing up and applying
sigmoid activation function

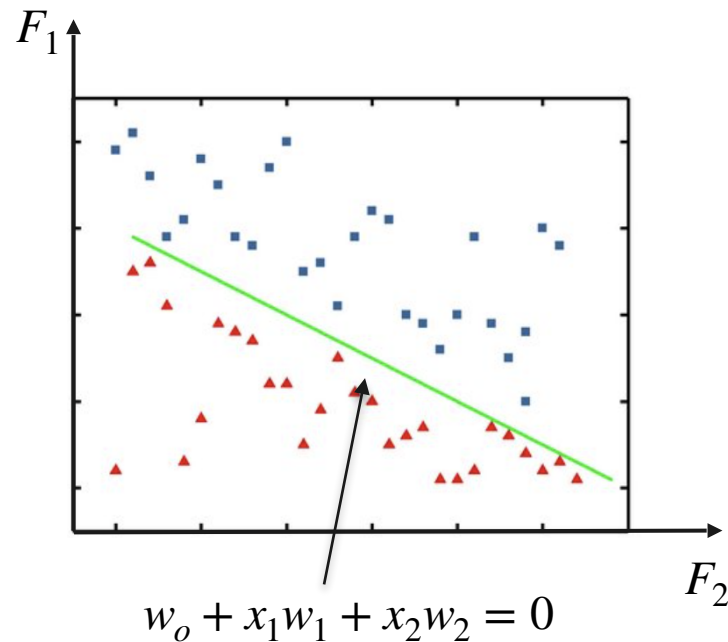
Logistic Regression

Logistic Regression is a weak algorithm

In case if we have only two features in our data:

	F_1	F_2	y
Object 1	0.1	1.34	0
Object 2	1.0	2.3	1
Object 3	0.98	1.24	1
Object 4	2.31	0.56	0

$$p = \sigma(w_o + x_1w_1 + x_2w_2)$$



The part of the formula inside the sigmoid function from a geometric point of view defines a **linear hyperplane**.

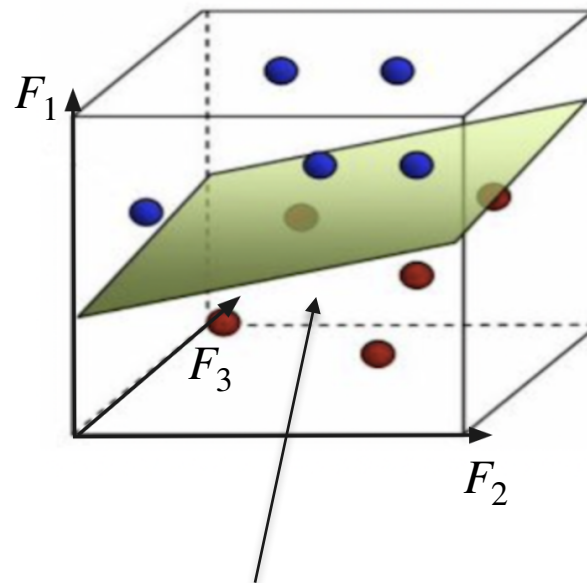
Logistic Regression

Logistic Regression is a weak algorithm

In case of 3-dimensional data:

	F_1	F_2	F_3	y
Object 1	0.1	1.34	3.45	0
Object 2	1.0	2.3	- 4.5	1
Object 3	0.98	1.24	- 1.34	1
Object 4	2.31	0.56	- 0.78	0

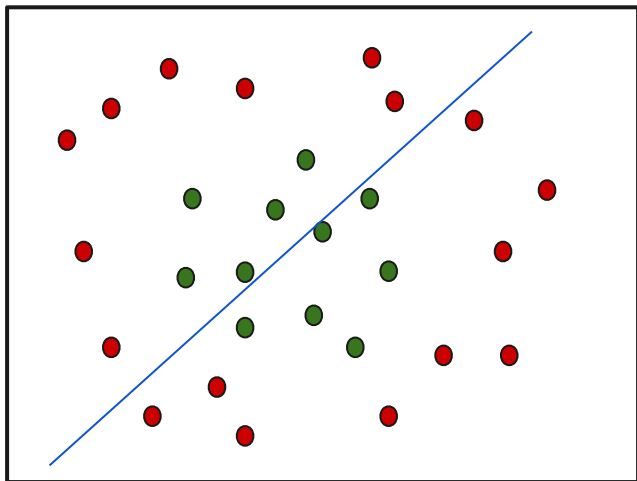
$$p = \sigma(w_o + x_1w_1 + x_2w_2)$$



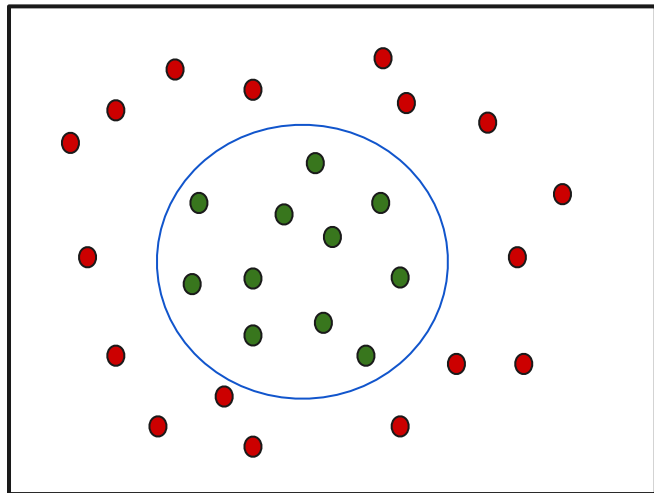
$$w_o + x_1w_1 + x_2w_2 + x_3w_3 = 0$$

Logistic Regression

Logistic Regression fits badly if the data is linearly inseparable



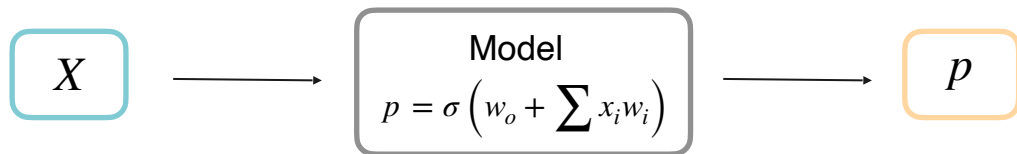
What we get



What we would like to have

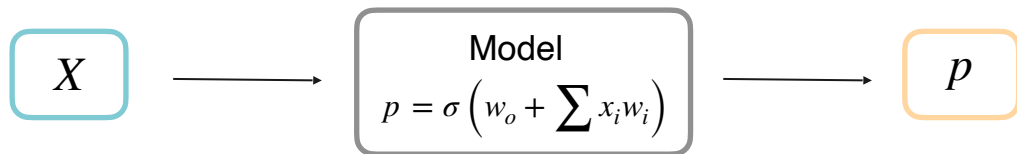
Making Logistic Regression better

Our current pipeline:

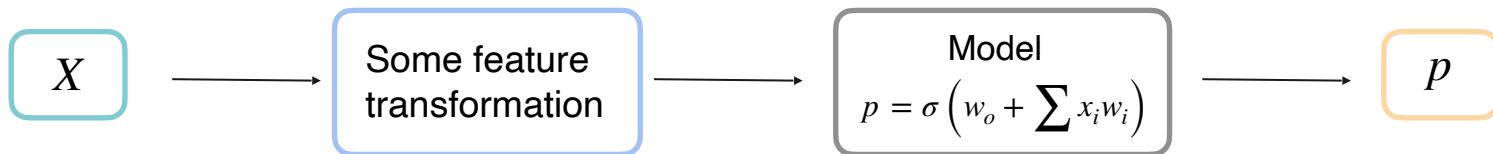


Making Logistic Regression better

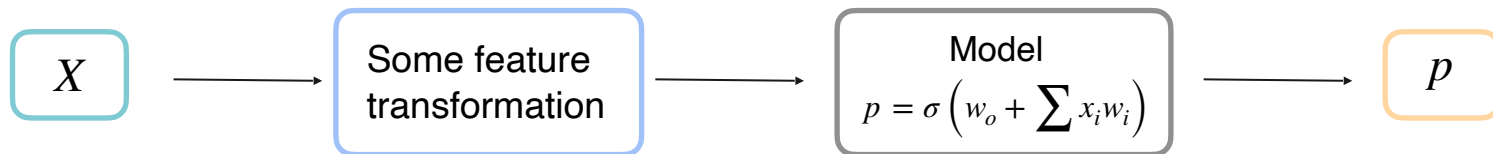
Our current pipeline:



Before feeding the object's features as input to the model, let us try to modify these features in a way so that they become linearly separable.



Making Logistic Regression better

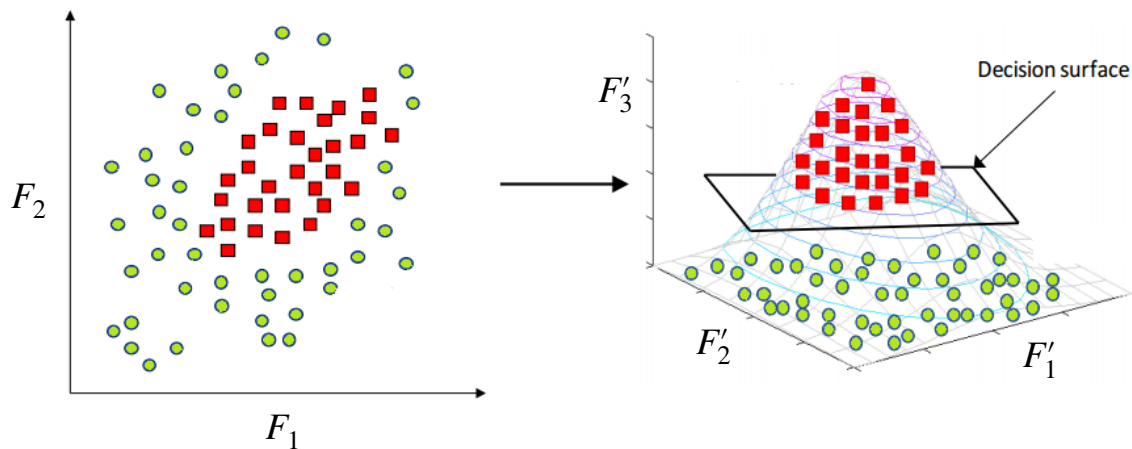


This idea is not new.

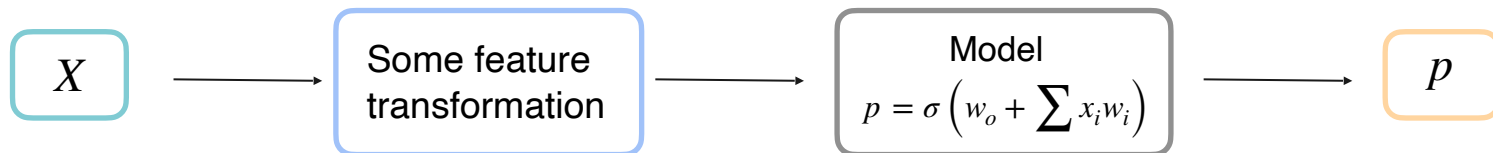
One of the ways to do this is using **kernels** (kernel trick)

Kernels are functions that modify features so that they become linearly separable

Dimension of features may change

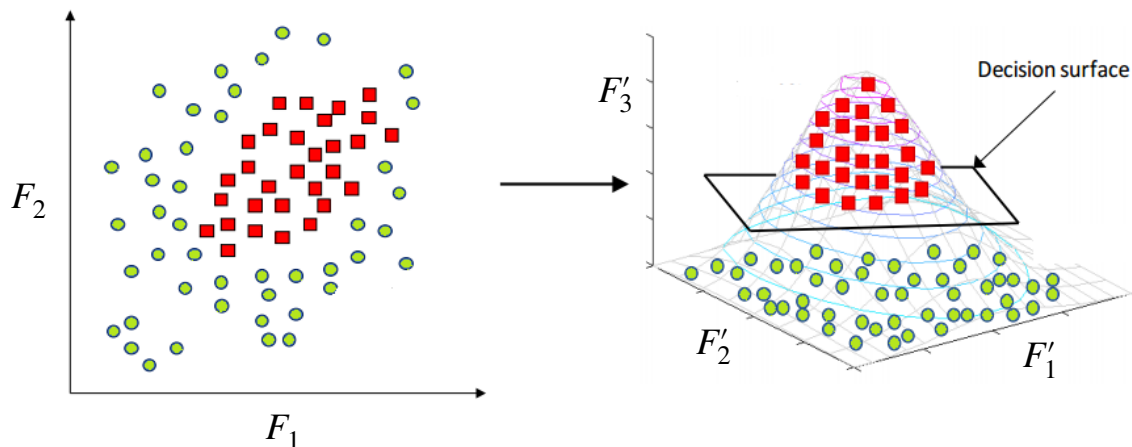


Making Logistic Regression better

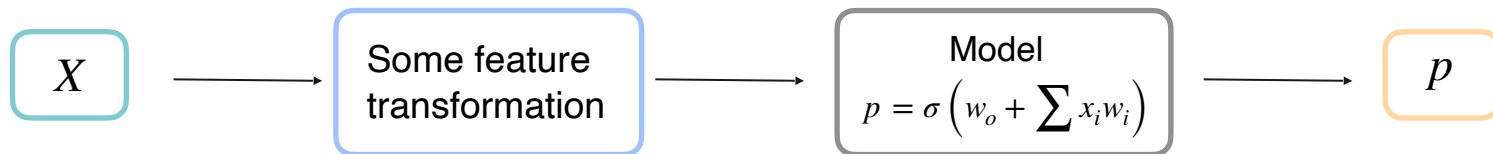


Kernels are functions of a fixed form.
They do not always work well

We want a more clever way of transforming features



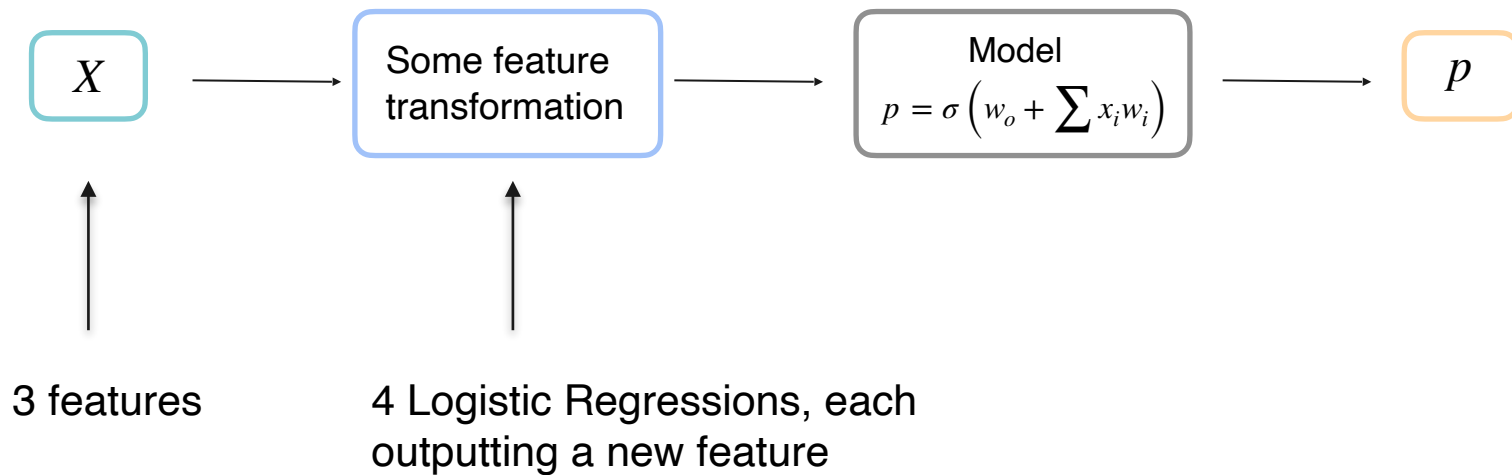
Making Logistic Regression better



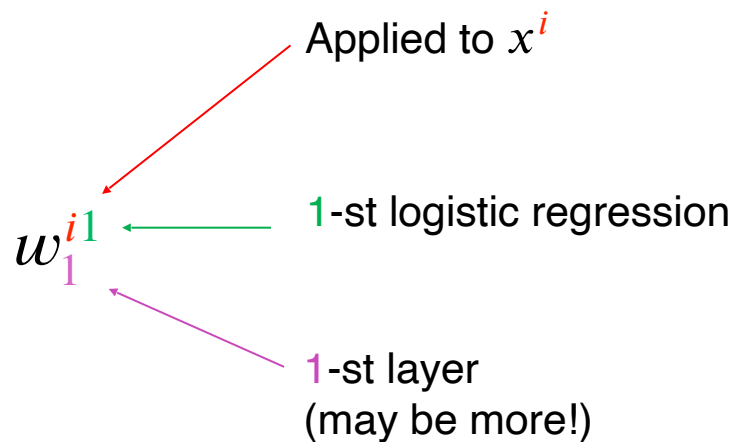
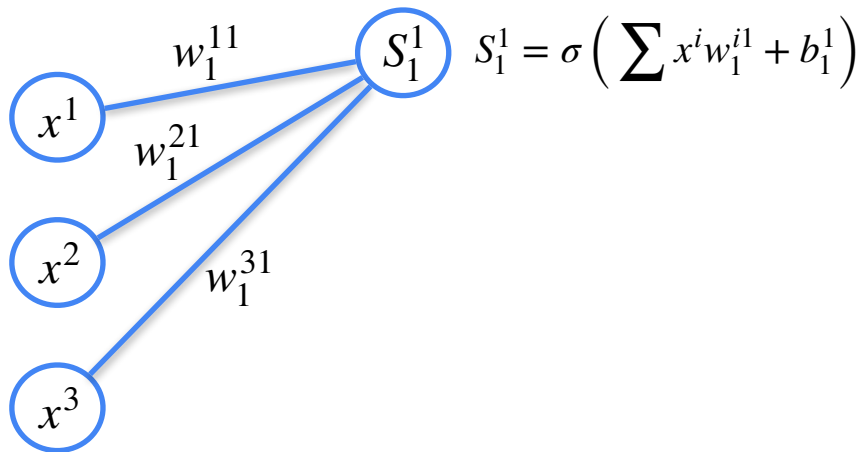
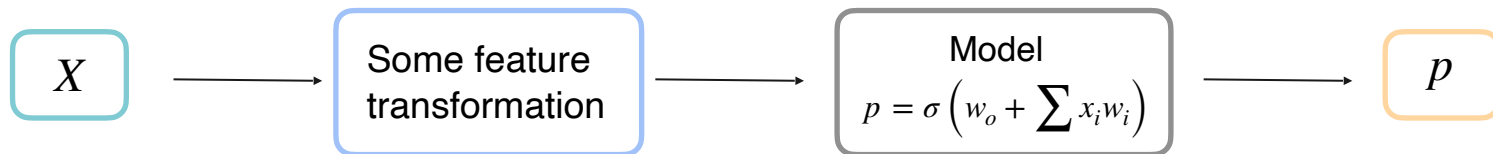
Idea: **what if we make our feature transformation module trainable** as well?

Then we could train on a certain dataset so that transformed features become linearly separable.

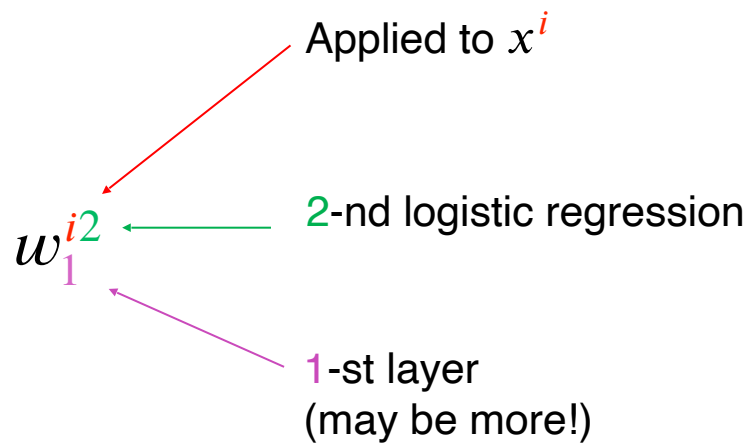
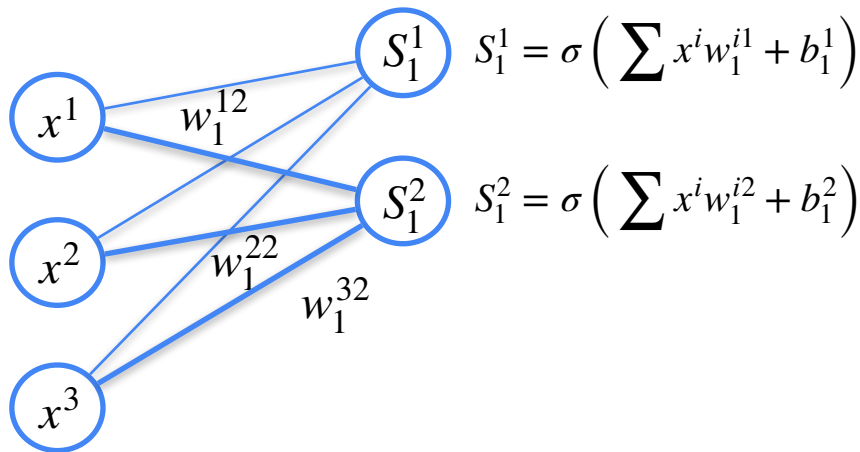
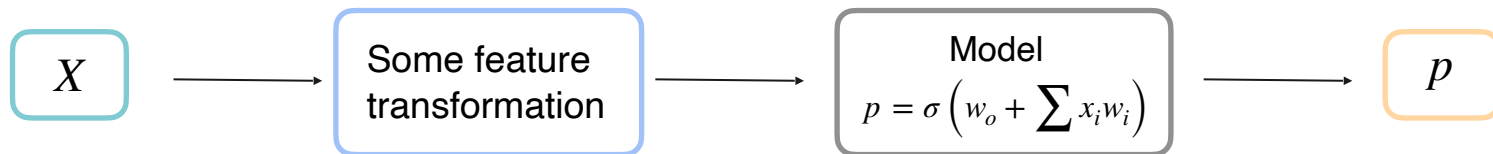
Making Logistic Regression better



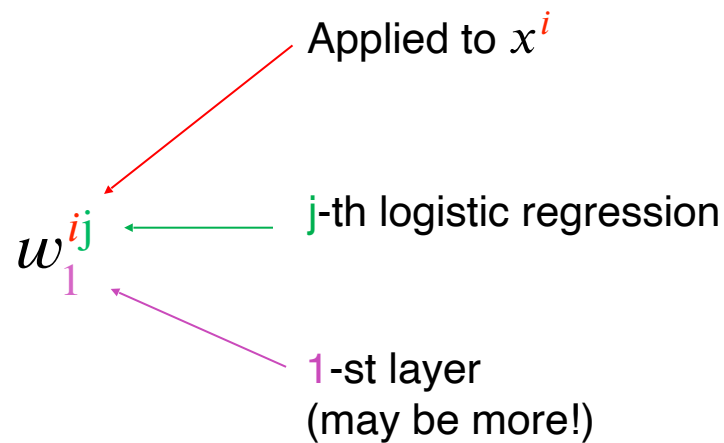
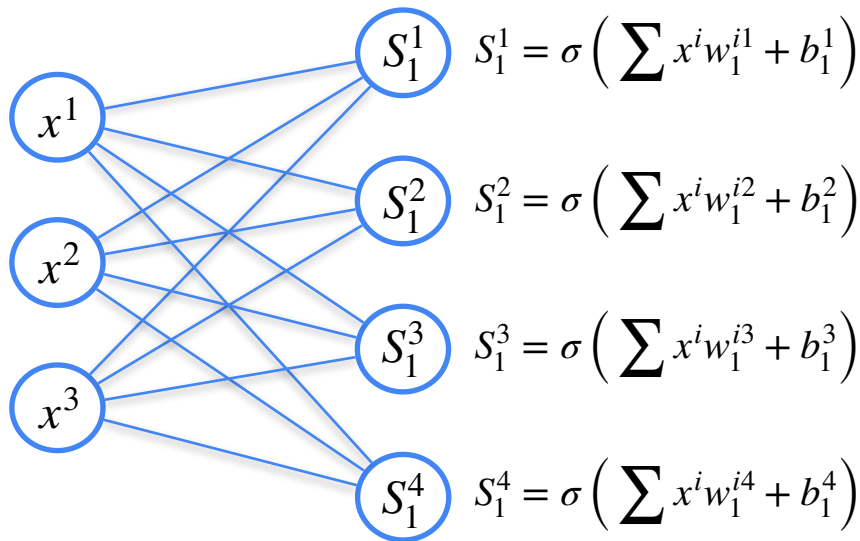
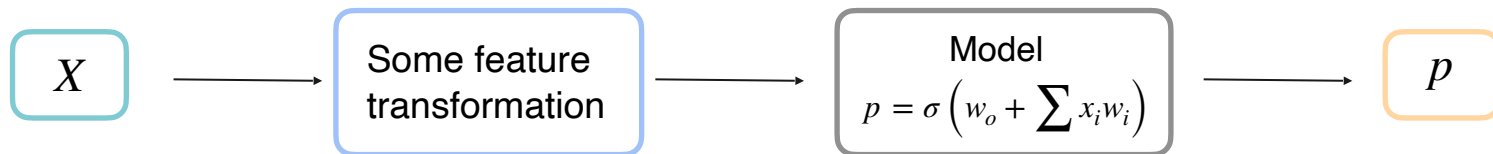
Making Logistic Regression better



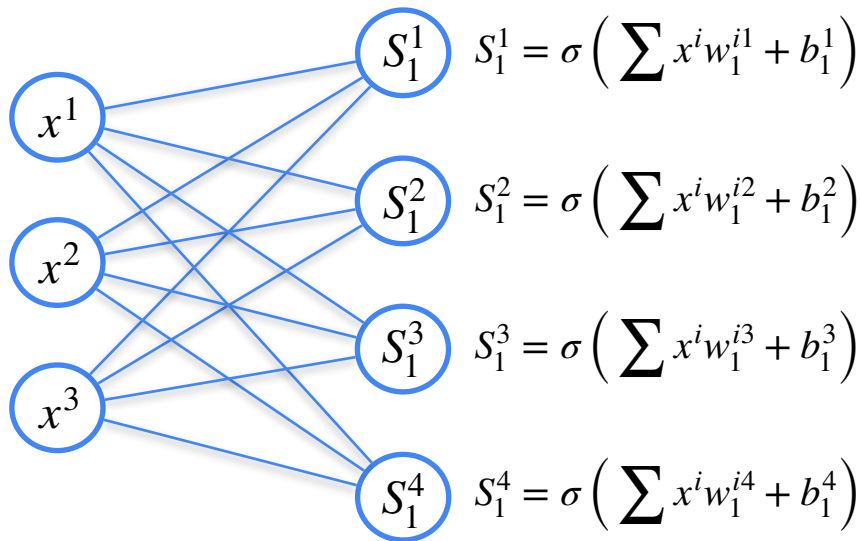
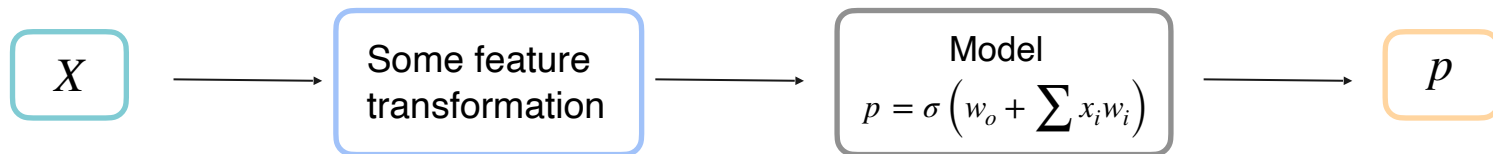
Making Logistic Regression better



Making Logistic Regression better



Making Logistic Regression better



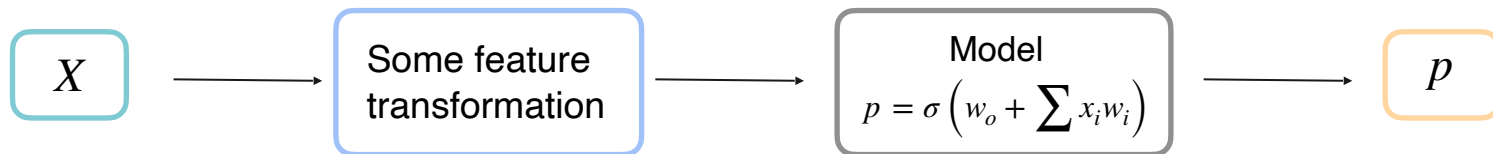
$$x = (x^1 \quad x^2 \quad x^3)$$

$$b_1 = (b_1^1 \quad b_1^2 \quad b_1^3 \quad b_1^4)$$

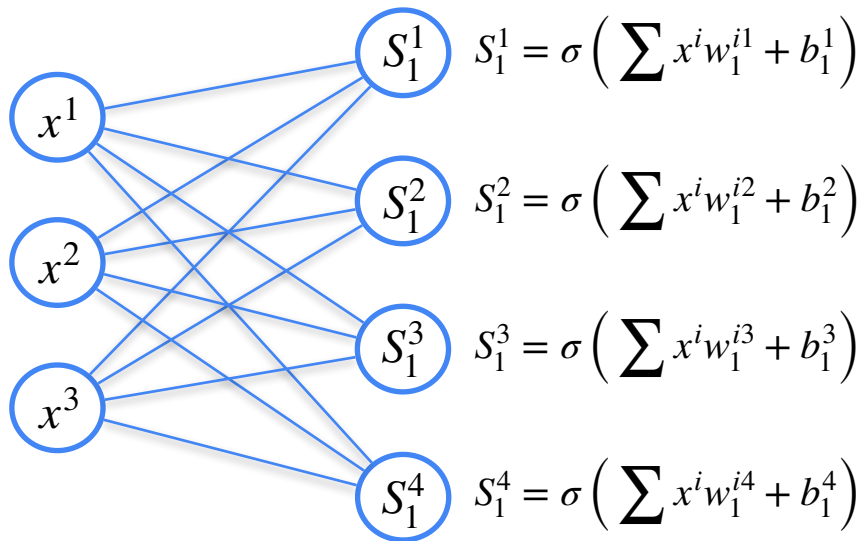
$$S_1 = (S_1^1 \quad S_1^2 \quad S_1^3 \quad S_1^4)$$

$$W_1 = \begin{pmatrix} w_1^{11} & w_1^{12} & w_1^{13} & w_1^{14} \\ w_1^{21} & w_1^{22} & w_1^{23} & w_1^{24} \\ w_1^{31} & w_1^{32} & w_1^{33} & w_1^{34} \end{pmatrix}$$

Making Logistic Regression better



Weights of 1st model S_1^1



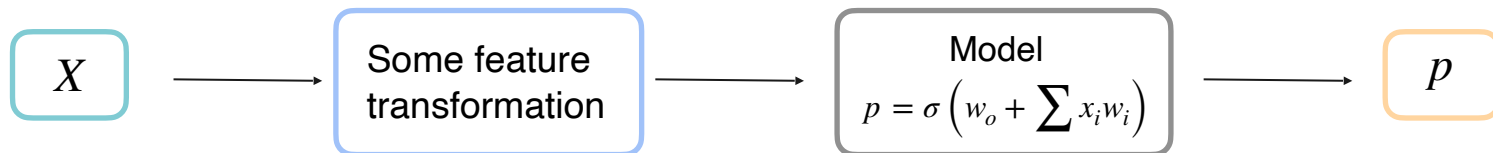
$$x = (x^1 \quad x^2 \quad x^3)$$

$$b_1 = (b_1^1 \quad b_1^2 \quad b_1^3 \quad b_1^4)$$

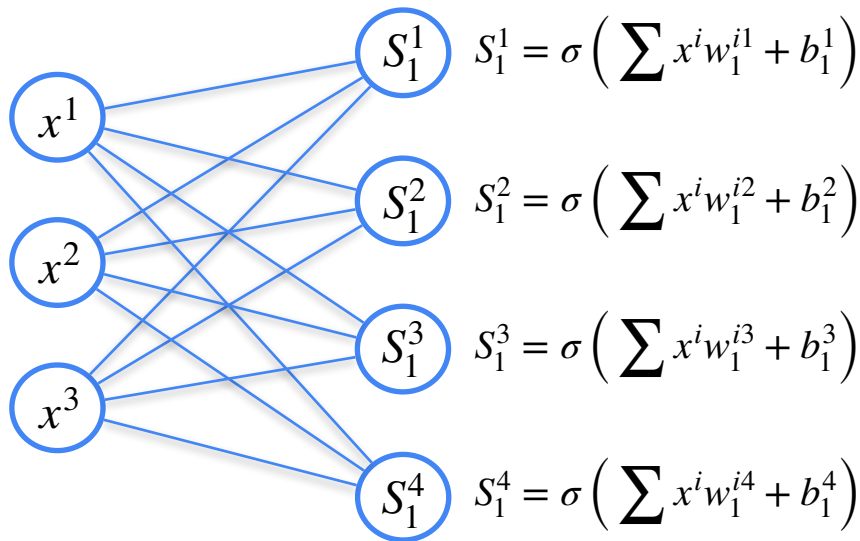
$$S_1 = (S_1^1 \quad S_1^2 \quad S_1^3 \quad S_1^4)$$

$$W_1 = \begin{pmatrix} w_1^{11} & w_1^{12} & w_1^{13} & w_1^{14} \\ w_1^{21} & w_1^{22} & w_1^{23} & w_1^{24} \\ w_1^{31} & w_1^{32} & w_1^{33} & w_1^{34} \end{pmatrix}$$

Making Logistic Regression better



Weights of 2nd model S_1^2



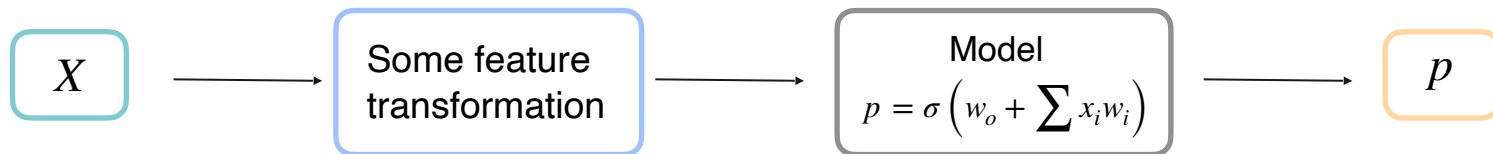
$$x = (x^1 \quad x^2 \quad x^3)$$

$$b_1 = (b_1^1 \quad b_1^2 \quad b_1^3 \quad b_1^4)$$

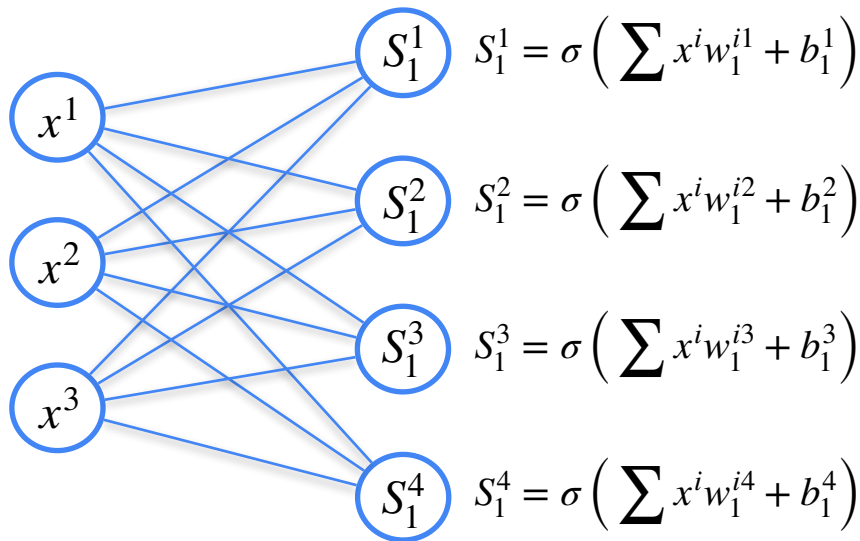
$$S_1 = (S_1^1 \quad S_1^2 \quad S_1^3 \quad S_1^4)$$

$$W_1 = \begin{pmatrix} w_1^{11} & w_1^{12} & w_1^{13} & w_1^{14} \\ w_1^{21} & w_1^{22} & w_1^{23} & w_1^{24} \\ w_1^{31} & w_1^{32} & w_1^{33} & w_1^{34} \end{pmatrix}$$

Making Logistic Regression better



Weights of 3rd model S_1^3



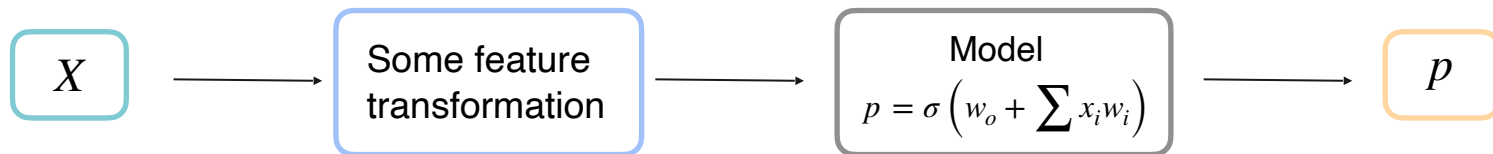
$$x = (x^1 \quad x^2 \quad x^3)$$

$$b_1 = (b_1^1 \quad b_1^2 \quad b_1^3 \quad b_1^4)$$

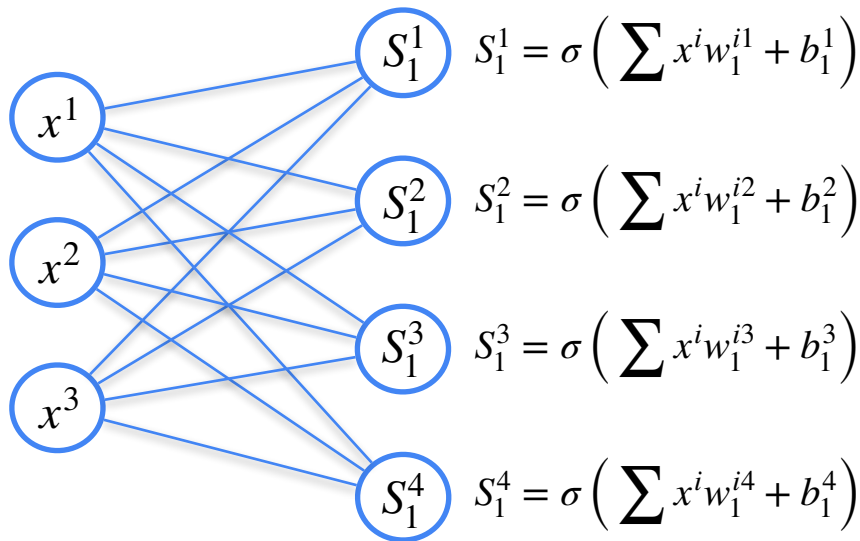
$$S_1 = (S_1^1 \quad S_1^2 \quad S_1^3 \quad S_1^4)$$

$$W_1 = \begin{pmatrix} w_1^{11} & w_1^{12} & w_1^{13} & w_1^{14} \\ w_1^{21} & w_1^{22} & w_1^{23} & w_1^{24} \\ w_1^{31} & w_1^{32} & w_1^{33} & w_1^{34} \end{pmatrix}$$

Making Logistic Regression better



Weights of 4th model S_1^4



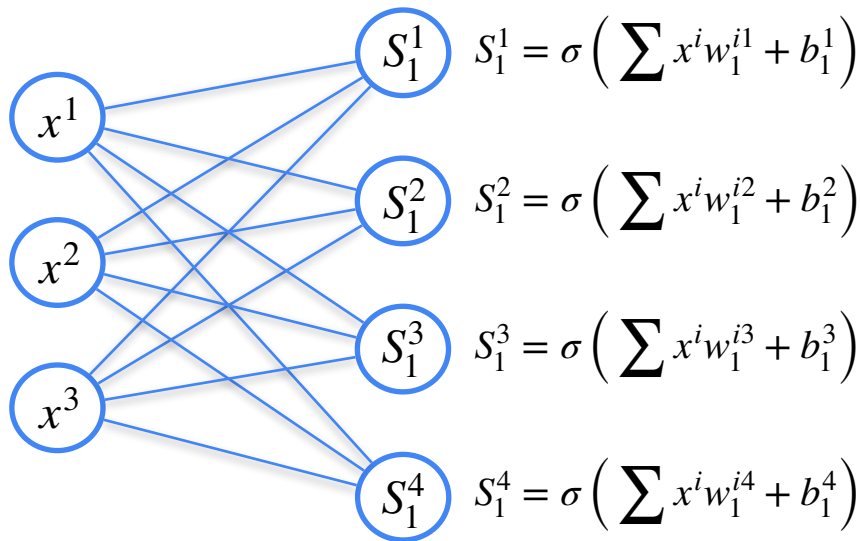
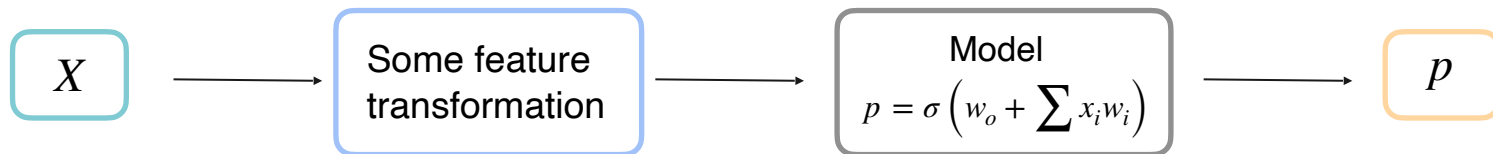
$$x = (x^1 \ x^2 \ x^3)$$

$$b_1 = (b_1^1 \ b_1^2 \ b_1^3 \ b_1^4)$$

$$S_1 = (S_1^1 \ S_1^2 \ S_1^3 \ S_1^4)$$

$$W_1 = \begin{pmatrix} w_1^{11} & w_1^{12} & w_1^{13} & w_1^{14} \\ w_1^{21} & w_1^{22} & w_1^{23} & w_1^{24} \\ w_1^{31} & w_1^{32} & w_1^{33} & w_1^{34} \end{pmatrix}$$

Making Logistic Regression better

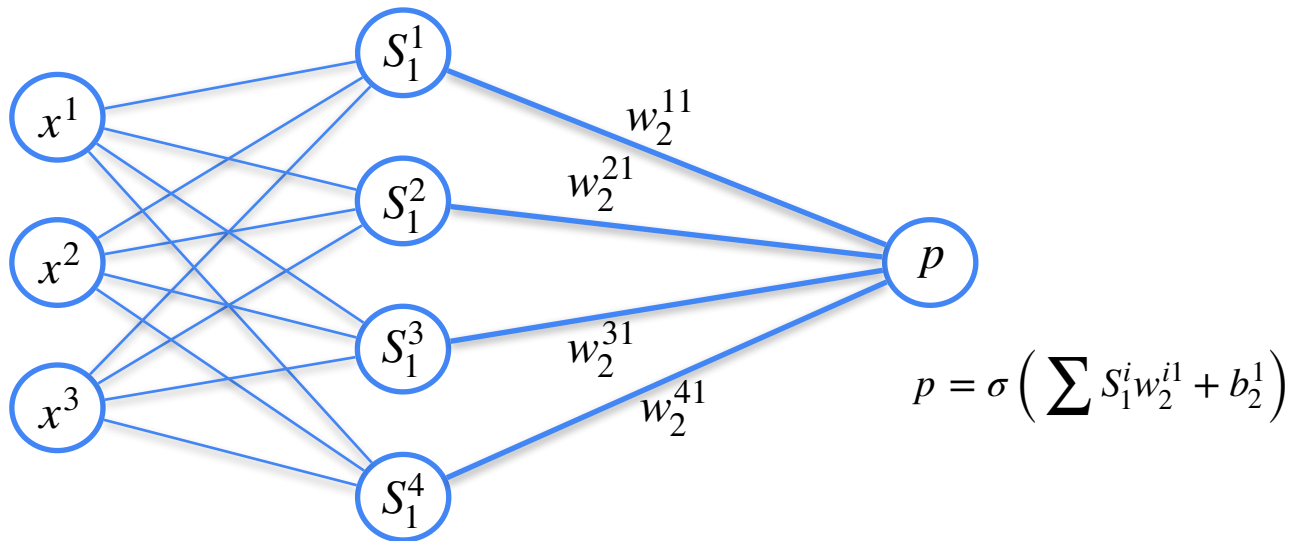
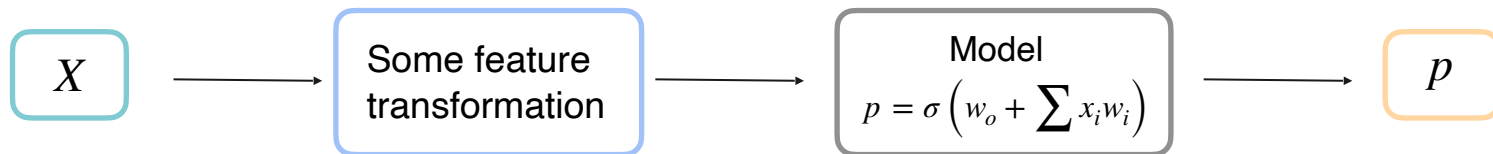


$$x = (x^1 \quad x^2 \quad x^3)$$
$$b_1 = (b_1^1 \quad b_1^2 \quad b_1^3 \quad b_1^4)$$
$$S_1 = (S_1^1 \quad S_1^2 \quad S_1^3 \quad S_1^4)$$
$$W_1 = \begin{pmatrix} w_1^{11} & w_1^{12} & w_1^{13} & w_1^{14} \\ w_1^{21} & w_1^{22} & w_1^{23} & w_1^{24} \\ w_1^{31} & w_1^{32} & w_1^{33} & w_1^{34} \end{pmatrix}$$

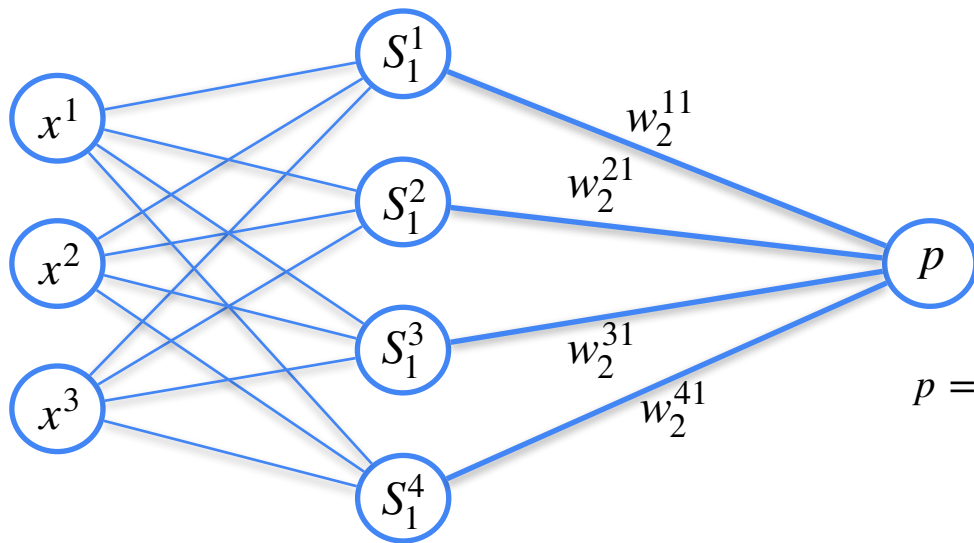
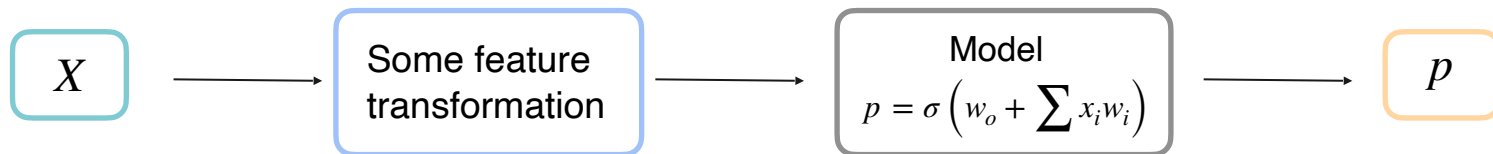
Formula for obtaining new features:

$$S_1 = \sigma (X W_1 + b_1)$$

Making Logistic Regression better



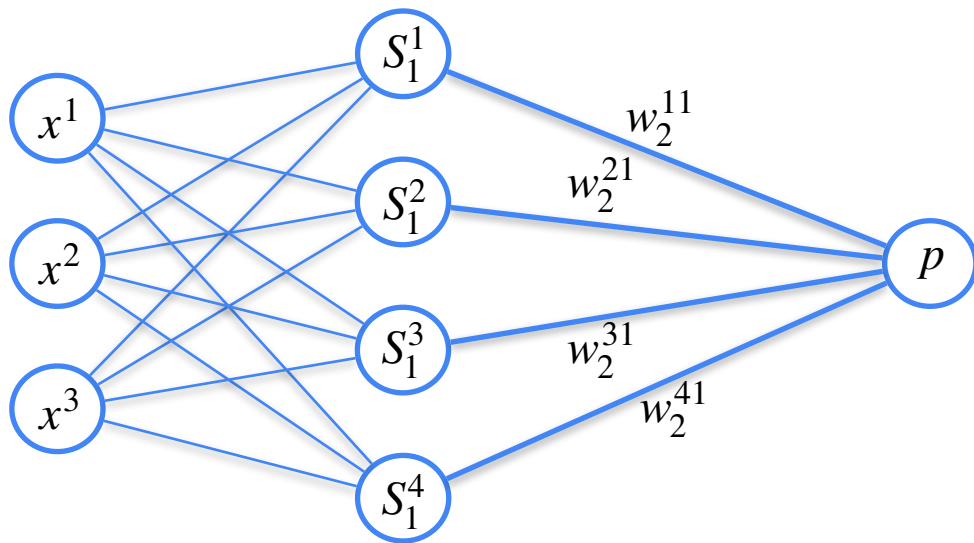
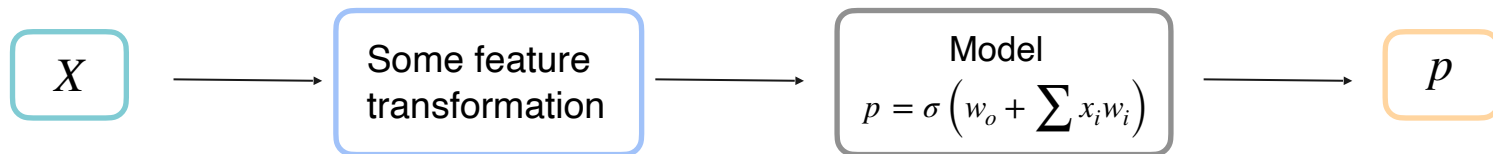
Making Logistic Regression better



$$S_1 = (S_1^1 \quad S_1^2 \quad S_1^3 \quad S_1^4)$$
$$b_2 = (b_2^1)$$
$$W_2 = \begin{pmatrix} w_2^{11} \\ w_2^{21} \\ w_2^{31} \\ w_2^{41} \end{pmatrix}$$

$$p = \sigma \left(\sum S_1^i w_2^{i1} + b_2^1 \right)$$

Making Logistic Regression better

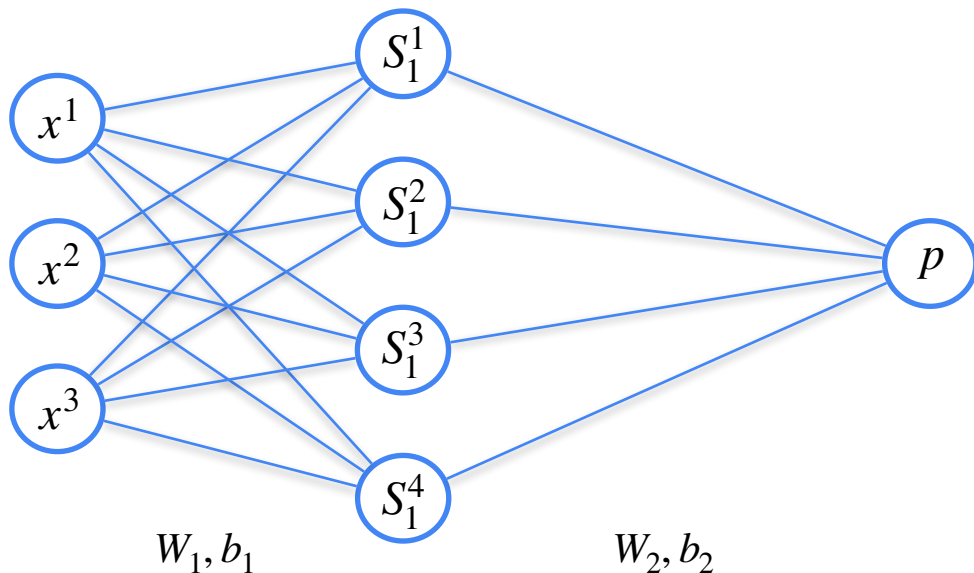
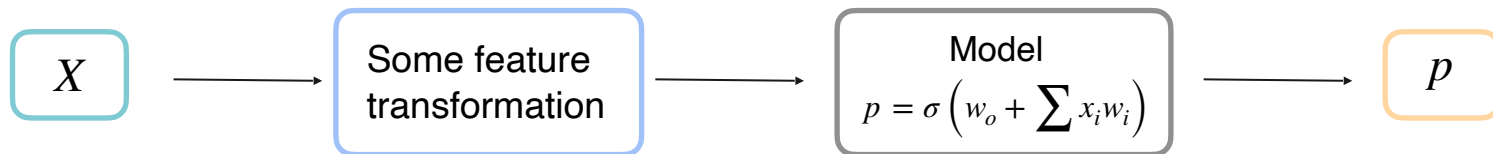


$$S_1 = (S_1^1 \quad S_1^2 \quad S_1^3 \quad S_1^4)$$
$$b_2 = (b_2^1)$$
$$W_2 = \begin{pmatrix} w_2^{11} \\ w_2^{21} \\ w_2^{31} \\ w_2^{41} \end{pmatrix}$$

Formula for obtaining output:

$$p = \sigma(S_1 W_2 + b_2)$$

Making Logistic Regression better



Formula for new features:

$$S_1 = \sigma(XW_1 + b_1)$$

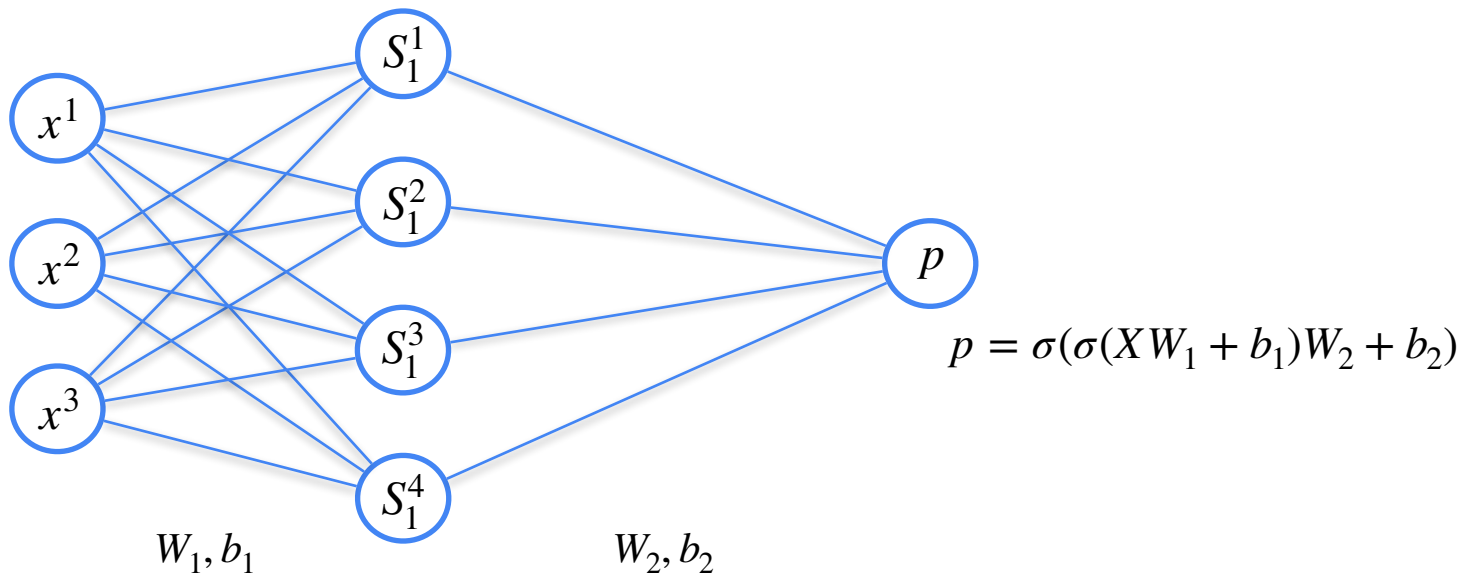
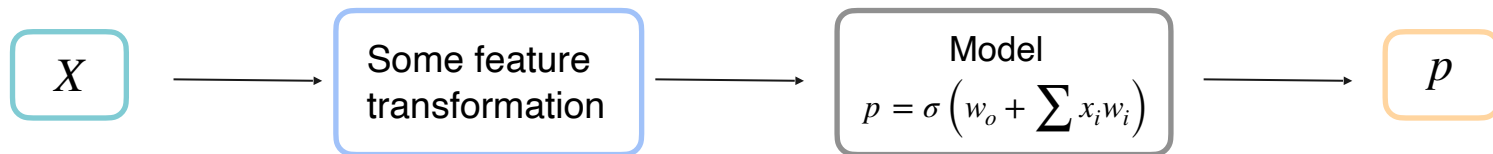
Formula for output:

$$p = \sigma(S_1W_2 + b_2)$$

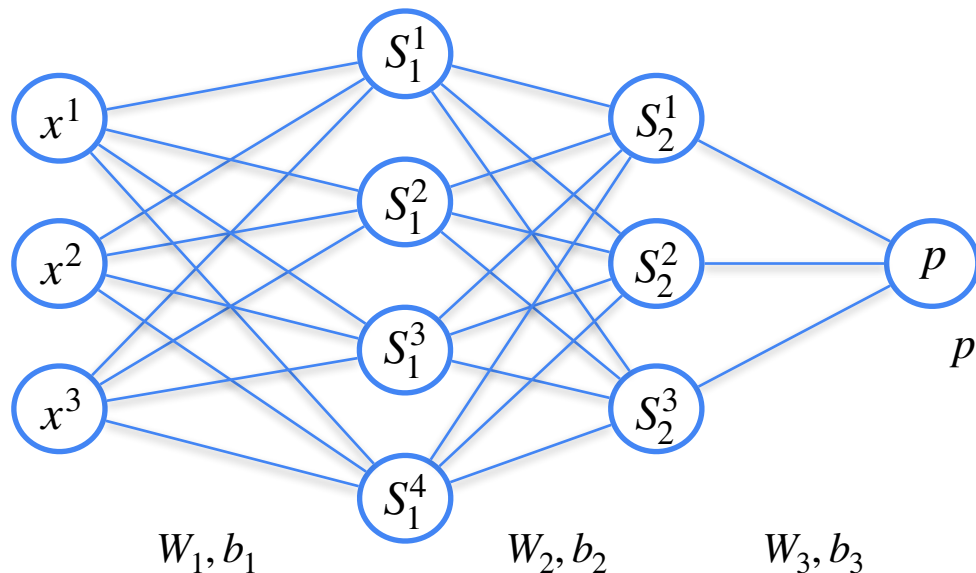
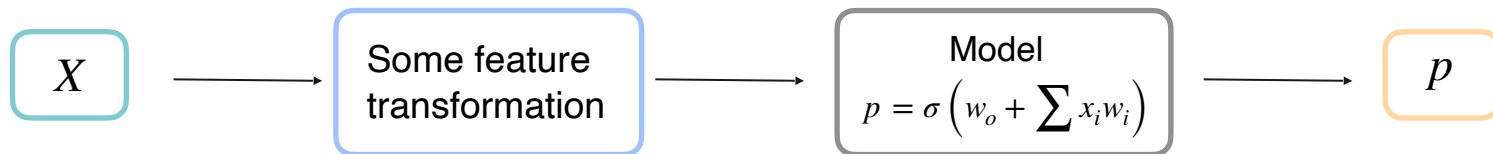
Plugging in:

$$p = \sigma(\sigma(XW_1 + b_1)W_2 + b_2)$$

Making Logistic Regression better



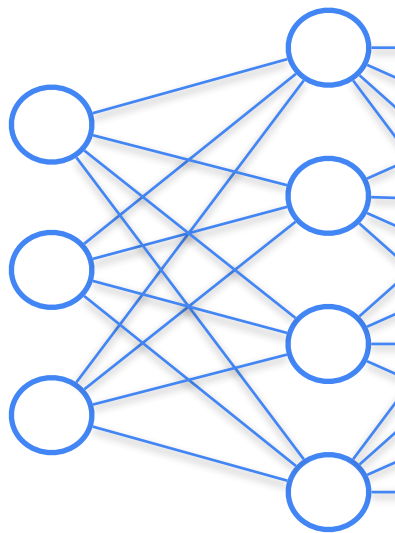
Making Logistic Regression better



We can have more layers of transformations

$$p = \sigma(\sigma(\sigma(XW_1 + b_1)W_2 + b_2)W_3 + b_3)$$

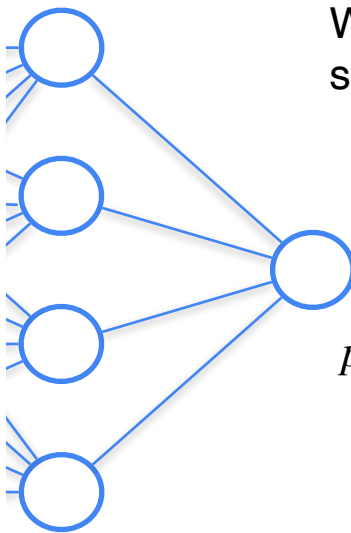
Fully-connected Neural Network



X

S_1

W_1, b_1



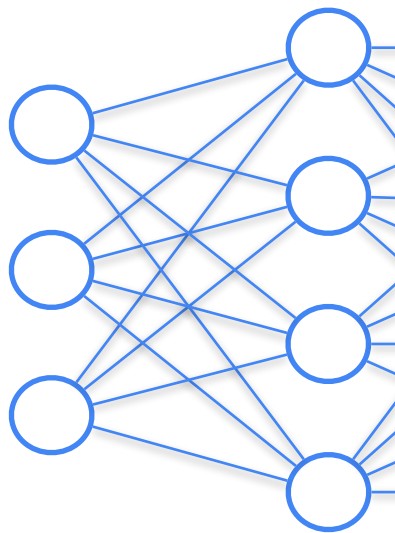
S_{m-1}

W_m, b_m

We have built a neural network for solving binary classification task

$$p = \sigma(\dots\sigma(XW_1 + b_1)\dots)W_m + b_m$$

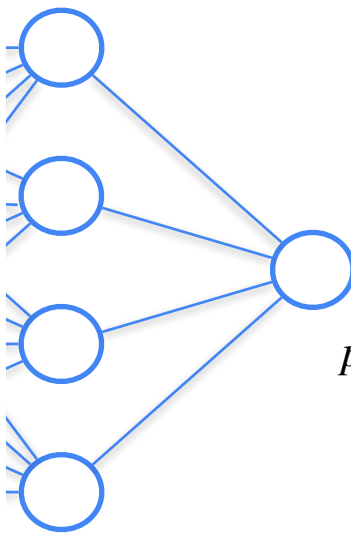
Fully-connected Neural Network



X

S_1

W_1, b_1



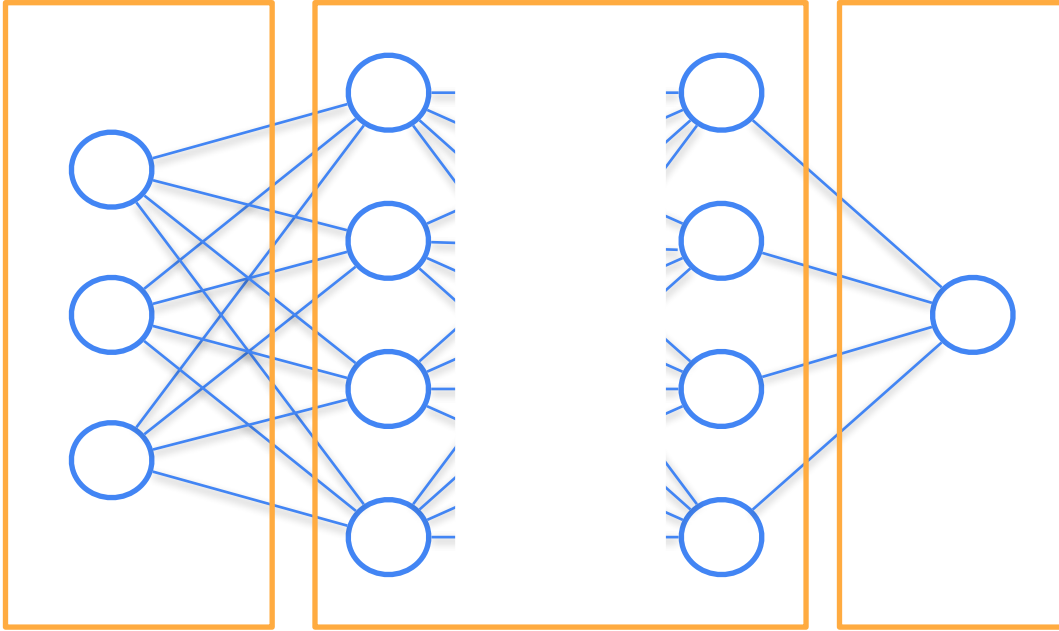
S_{m-1}

W_m, b_m

For regression task we can
change the last layer to represent
Linear Regression

$$p = S_{m-1}W_m + b_m$$

Fully-connected Neural Network



Input layer

hidden layers

output

Progress

What we have so far:

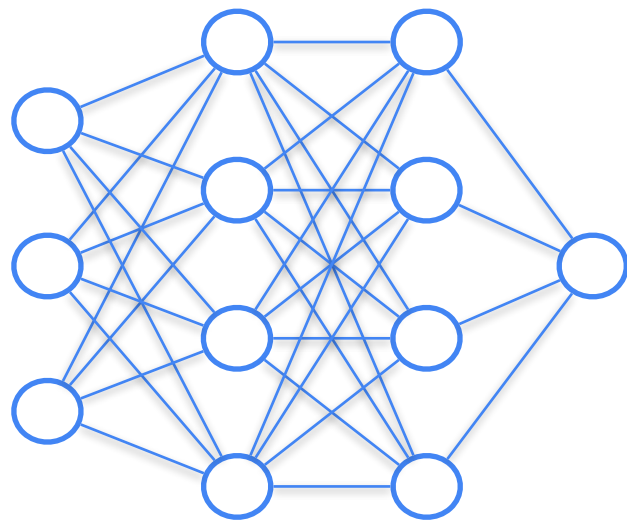
- Structure of fully-connected Network derived from Logistic Regression and kernel ideas
- We can create neural networks for binary classification and regression tasks

Next:

- Training of neural networks
- Activation functions beyond sigmoid
- Building neural network on PyTorch
- Multi-class classification task
- Additional types of layers, regularisation

Training of Neural Networks

Fully-connected Neural Network



X S_1 S_2 p

W_1, b_1

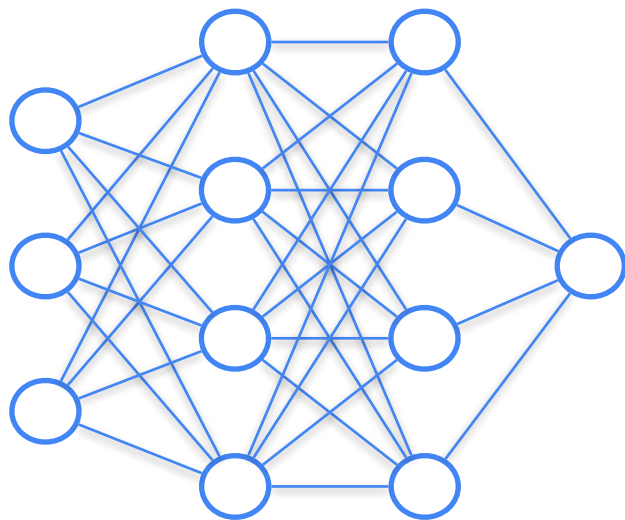
W_2, b_2

W_3, b_3

$$p = \sigma(\sigma(\sigma(XW_1 + b_1)W_2 + b_2)W_3 + b_3)$$

Neural network is a function of its parameters

Fully-connected Neural Network



X S_1 S_2 p

W_1, b_1 W_2, b_2 W_3, b_3

$$p = \sigma(\sigma(\sigma(XW_1 + b_1)W_2 + b_2)W_3 + b_3)$$

Neural network is a function of its parameters

Let's choose the loss function for training our NN:

$$loss = L(y, p) \rightarrow \min$$

Fully-connected Neural Network

Neural network is a function of its parameters:

$$p = \sigma(\sigma(\sigma(XW_1 + b_1)W_2 + b_2)W_3 + b_3)$$

The loss function:

$$loss = L(y, p) \rightarrow min$$

Plugging formula for p in:

$$loss = L(y, \sigma(\sigma(\sigma(XW_1 + b_1)W_2 + b_2)W_3 + b_3)) \rightarrow min$$

Fully-connected Neural Network

Neural network is a function of its parameters:

$$p = \sigma(\sigma(\sigma(XW_1 + b_1)W_2 + b_2)W_3 + b_3)$$

The loss function:

$$loss = L(y, p) \rightarrow min$$

Plugging formula for p in:

$$loss = L(y, \sigma(\sigma(\sigma(XW_1 + b_1)W_2 + b_2)W_3 + b_3)) \rightarrow min$$

This is a function of W_i, b_i

We can optimise it using
gradient descent!

At each training step, we need to

calculate the values of $\frac{\partial L}{\partial W_i}, \frac{\partial L}{\partial b_i}$



Fully-connected Neural Network

$$loss = L(y, \sigma(\sigma(\sigma(XW_1 + b_1)W_2 + b_2)W_3 + b_3))$$

At each training step, we need to calculate the values of $\frac{\partial L}{\partial W_i}, \frac{\partial L}{\partial b_i}$

W_i, b_i are matrices, and $\frac{\partial L}{\partial W_i}, \frac{\partial L}{\partial b_i}$ are matrices of partial derivatives $\left\{ \frac{\partial L}{\partial w_i^{jk}} \right\}$ and $\left\{ \frac{\partial L}{\partial b_i^j} \right\}$

$$\frac{\partial L}{\partial W_i} = \begin{pmatrix} \frac{\partial L}{\partial w_i^{11}} & \frac{\partial L}{\partial w_i^{12}} & \cdots & \frac{\partial L}{\partial w_i^{1l}} \\ \frac{\partial L}{\partial w_i^{21}} & \frac{\partial L}{\partial w_i^{22}} & \cdots & \frac{\partial L}{\partial w_i^{2l}} \\ \cdots & \cdots & \cdots & \cdots \\ \frac{\partial L}{\partial w_i^{m1}} & \frac{\partial L}{\partial w_i^{m2}} & \cdots & \frac{\partial L}{\partial w_i^{ml}} \end{pmatrix} \quad \frac{\partial L}{\partial b_i} = \begin{pmatrix} \frac{\partial L}{\partial b_i^1} & \frac{\partial L}{\partial b_i^2} & \cdots & \frac{\partial L}{\partial b_i^m} \end{pmatrix}$$

Fully-connected Neural Network

$$loss = L(y, \sigma(\sigma(\sigma(XW_1 + b_1)W_2 + b_2)W_3 + b_3))$$

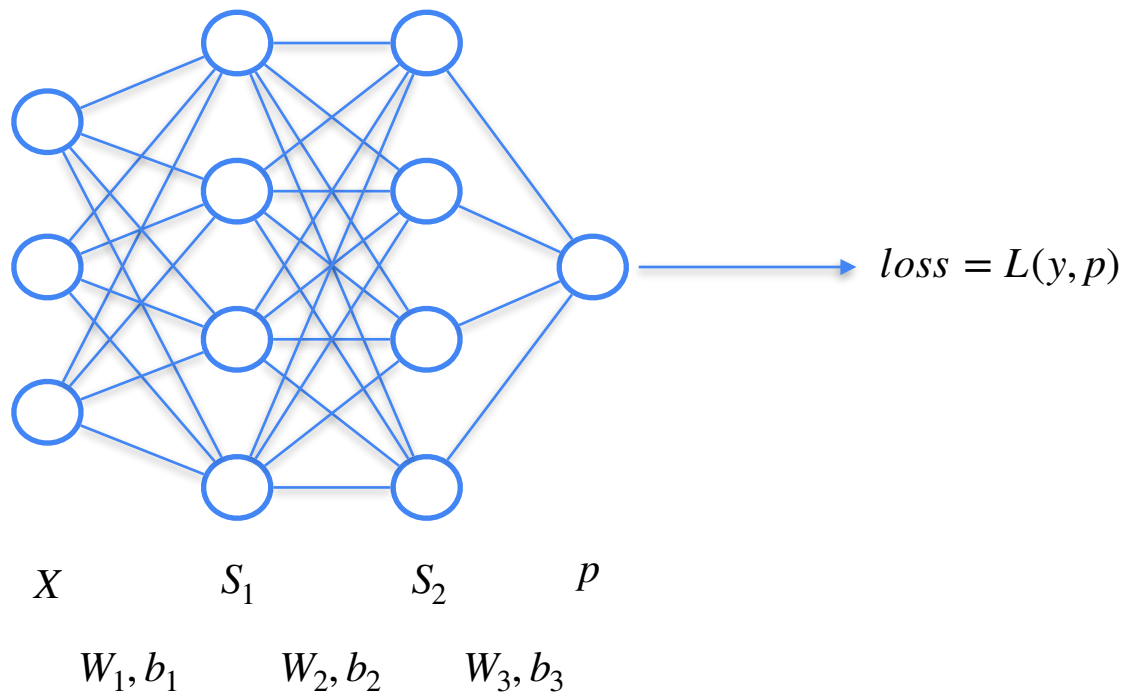
At each training step, we need to calculate the values of $\frac{\partial L}{\partial W_i}, \frac{\partial L}{\partial b_i}$

W_i, b_i are matrices, and $\frac{\partial L}{\partial W_i}, \frac{\partial L}{\partial b_i}$ are matrices of partial derivatives $\left\{ \frac{\partial L}{\partial w_i^{jk}} \right\}$ and $\left\{ \frac{\partial L}{\partial b_i^j} \right\}$

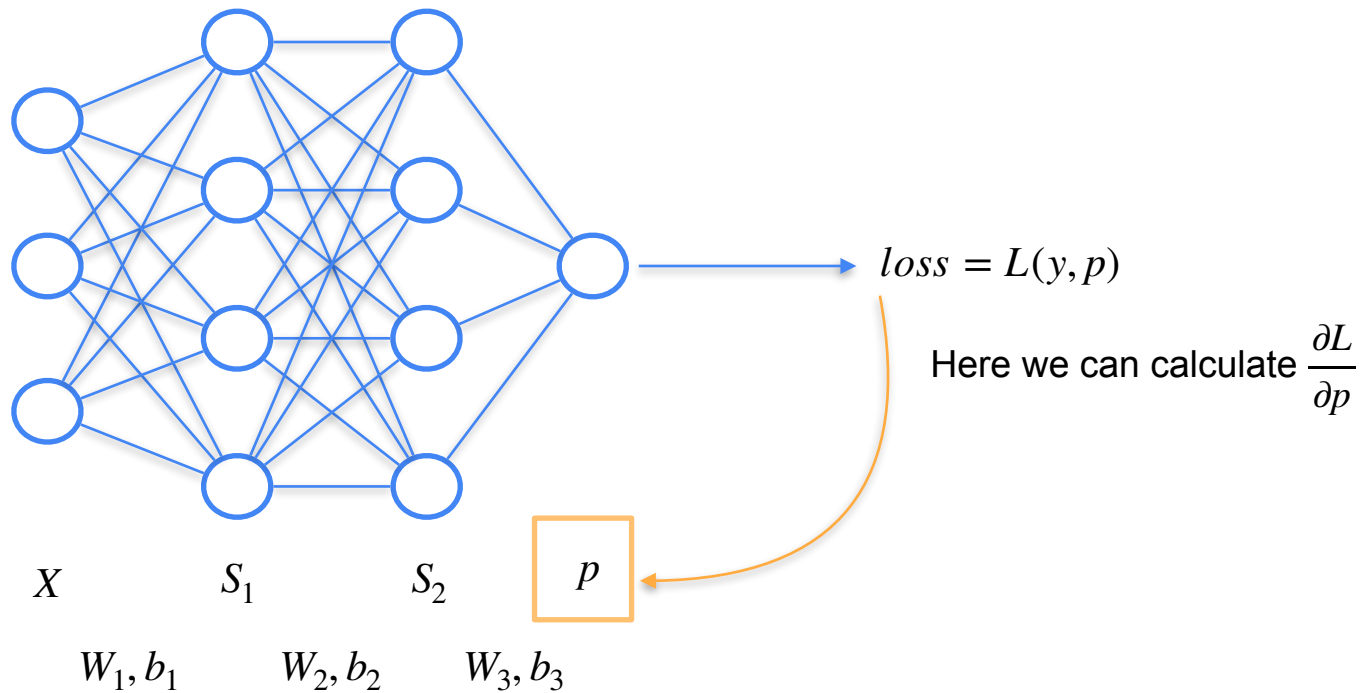
$$\frac{\partial L}{\partial W_i} = \begin{pmatrix} \frac{\partial L}{\partial w_i^{11}} & \frac{\partial L}{\partial w_i^{12}} & \cdots & \frac{\partial L}{\partial w_i^{1l}} \\ \frac{\partial L}{\partial w_i^{21}} & \frac{\partial L}{\partial w_i^{22}} & \cdots & \frac{\partial L}{\partial w_i^{2l}} \\ \cdots & \cdots & \cdots & \cdots \\ \frac{\partial L}{\partial w_i^{m1}} & \frac{\partial L}{\partial w_i^{m2}} & \cdots & \frac{\partial L}{\partial w_i^{ml}} \end{pmatrix} \quad \frac{\partial L}{\partial b_i} = \begin{pmatrix} \frac{\partial L}{\partial b_i^1} & \frac{\partial L}{\partial b_i^2} & \cdots & \frac{\partial L}{\partial b_i^m} \end{pmatrix}$$

We can calculate all the $\frac{\partial L}{\partial W_i}, \frac{\partial L}{\partial b_i}$ using rules of matrix derivatives!

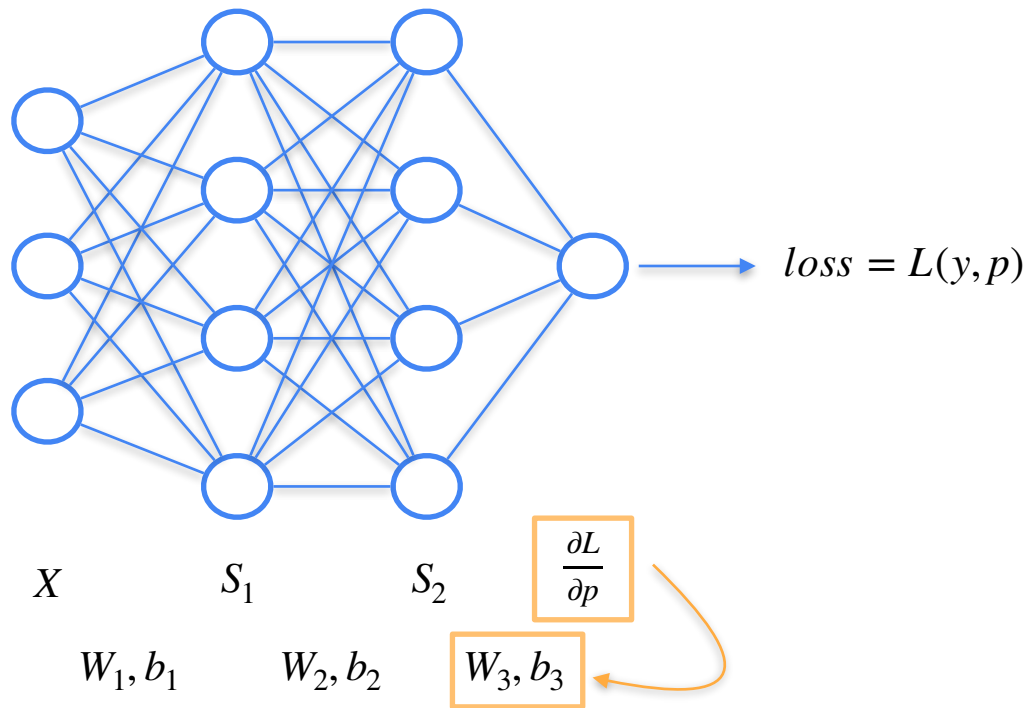
Fully-connected Neural Network



Fully-connected Neural Network



Fully-connected Neural Network



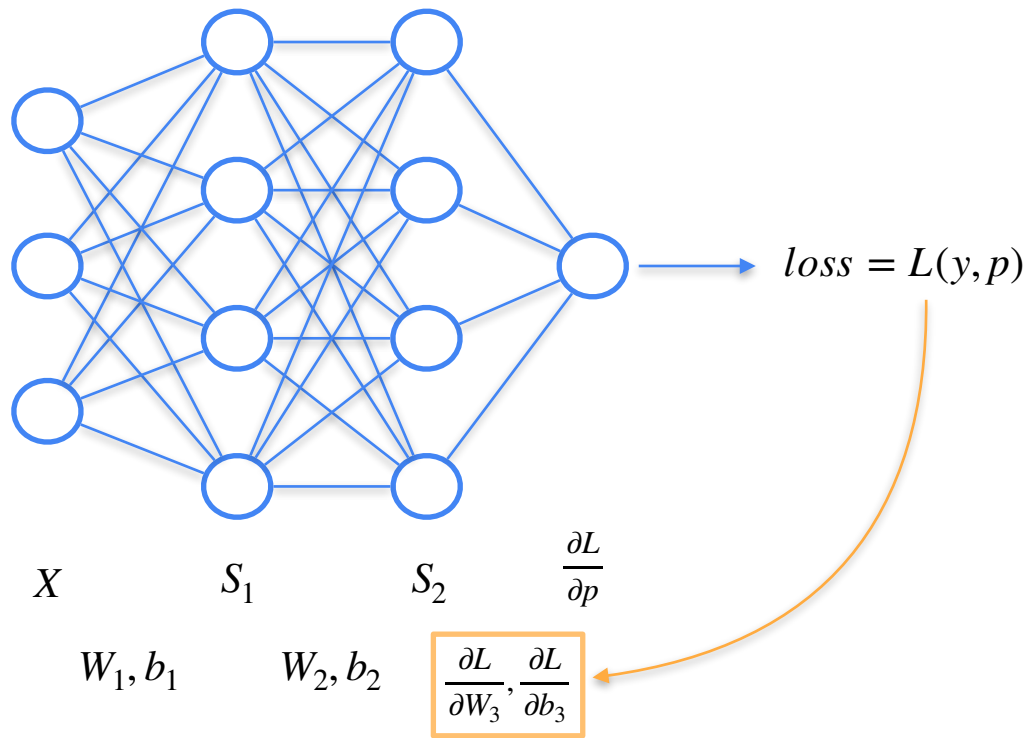
Next, we have $p = \sigma(\underbrace{S_2 W_3 + b_3}_{\hat{p}})$

We can calculate $\frac{\partial p}{\partial W_3}, \frac{\partial p}{\partial b_3}$

$$\frac{\partial p}{\partial W_3} = \frac{\partial p}{\partial \hat{p}} \frac{\partial \hat{p}}{\partial W_3}$$

$$\frac{\partial p}{\partial b_3} = \frac{\partial p}{\partial \hat{p}} \frac{\partial \hat{p}}{\partial b_3}$$

Fully-connected Neural Network



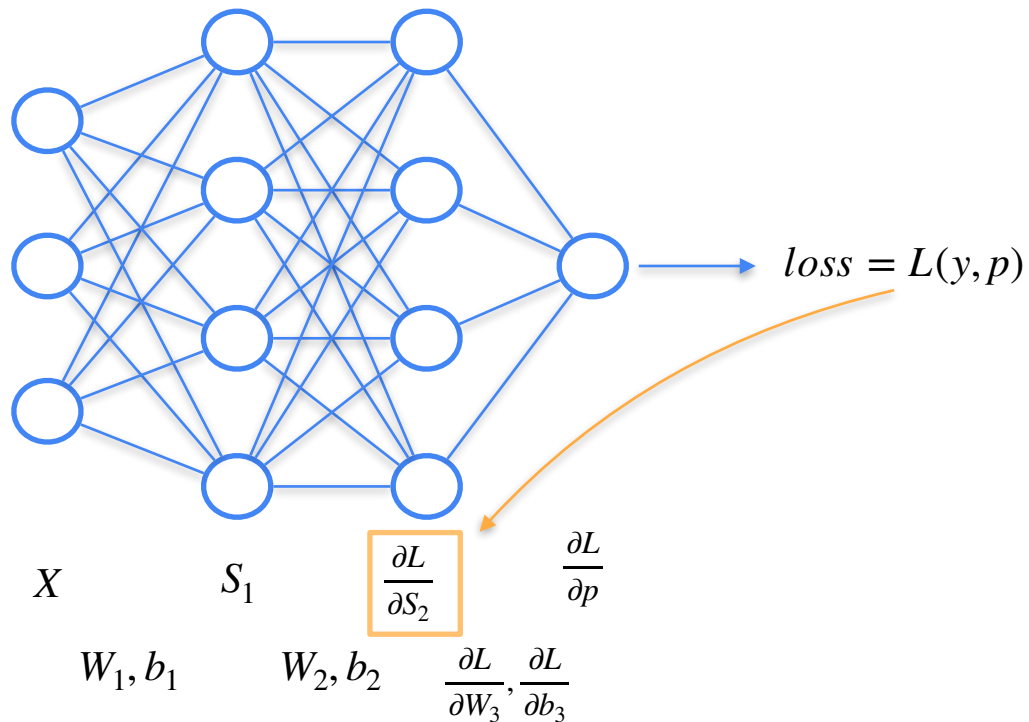
Next, we have $p = \sigma(\overbrace{S_2 W_3 + b_3}^{\hat{p}})$

Knowing $\frac{\partial L}{\partial p}$, we can now get $\frac{\partial L}{\partial W_3}, \frac{\partial L}{\partial b_3}$

$$\frac{\partial L}{\partial W_3} = \frac{\partial L}{\partial p} \frac{\partial p}{\partial \hat{p}} \frac{\partial \hat{p}}{\partial W_3}$$

$$\frac{\partial L}{\partial b_3} = \frac{\partial L}{\partial p} \frac{\partial p}{\partial \hat{p}} \frac{\partial \hat{p}}{\partial b_3}$$

Fully-connected Neural Network

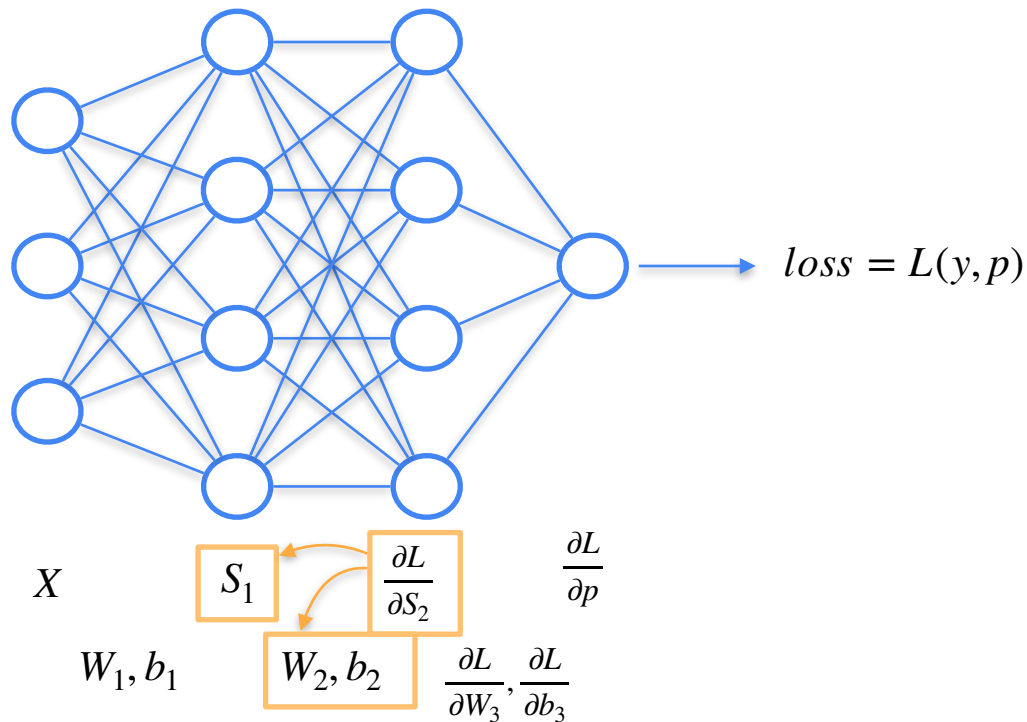


Next, we have $p = \sigma(\underbrace{S_2 W_3 + b_3}_{\hat{p}})$

We can also get $\frac{\partial L}{\partial S_2}$

$$\frac{\partial L}{\partial S_2} = \frac{\partial L}{\partial p} \frac{\partial p}{\partial \hat{p}} \frac{\partial \hat{p}}{\partial S_2}$$

Fully-connected Neural Network



Next, we have $S_2 = \sigma(\underbrace{S_1 W_2 + b_2}_{\widehat{S_2}})$

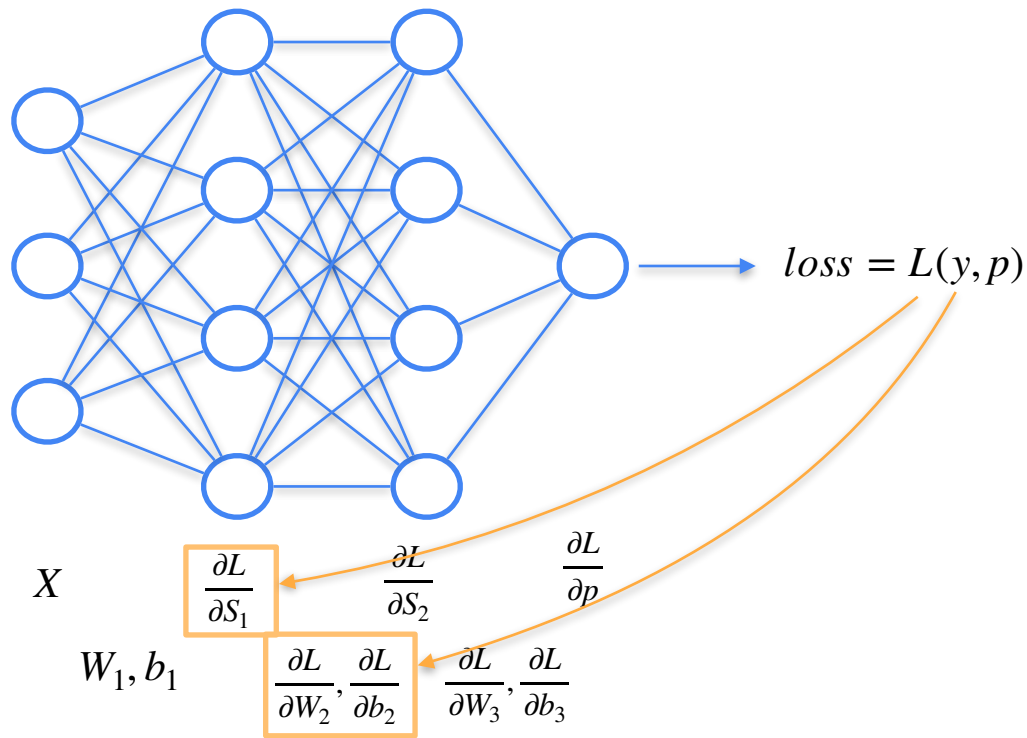
We can get $\frac{\partial S_2}{\partial W_2}, \frac{\partial S_2}{\partial b_2}, \frac{\partial S_2}{\partial S_1}$

$$\frac{\partial S_2}{\partial W_2} = \frac{\partial S_2}{\partial \widehat{S_2}} \frac{\partial \widehat{S_2}}{\partial W_2}$$

$$\frac{\partial S_2}{\partial b_2} = \frac{\partial S_2}{\partial \widehat{S_2}} \frac{\partial \widehat{S_2}}{\partial b_2}$$

$$\frac{\partial S_2}{\partial S_1} = \frac{\partial S_2}{\partial \widehat{S_2}} \frac{\partial \widehat{S_2}}{\partial S_1}$$

Fully-connected Neural Network



Next, we have $S_2 = \sigma(\underbrace{S_1 W_2 + b_2}_{\widehat{S}_2})$

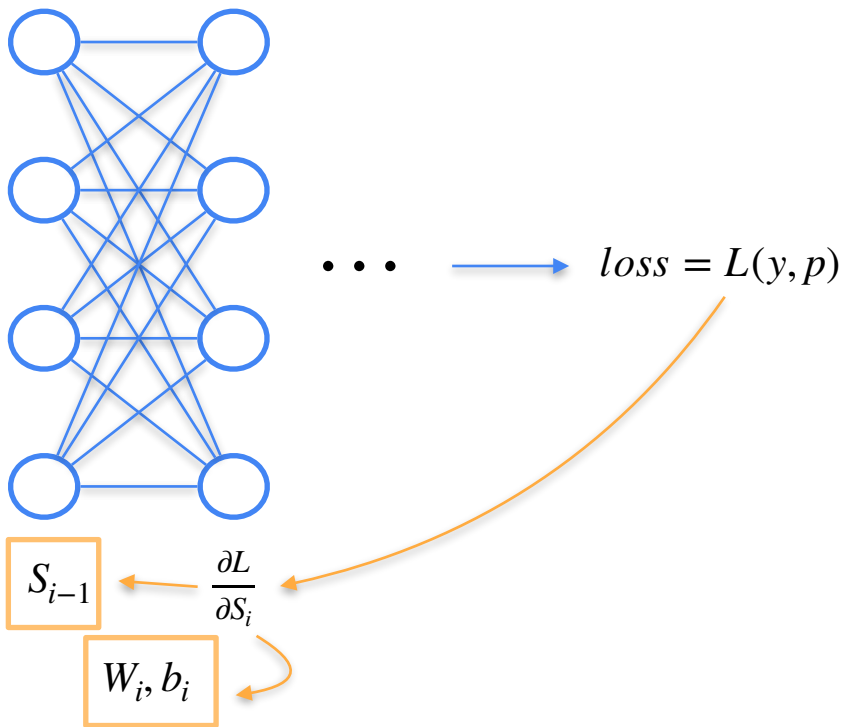
Combining $\frac{\partial S_2}{\partial W_2}$, $\frac{\partial S_2}{\partial b_2}$, $\frac{\partial S_2}{\partial S_1}$ and $\frac{\partial L}{\partial S_2}$, we get:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial S_2} \frac{\partial S_2}{\partial \widehat{S}_2} \frac{\partial \widehat{S}_2}{\partial W_2}$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial S_2} \frac{\partial S_2}{\partial \widehat{S}_2} \frac{\partial \widehat{S}_2}{\partial b_2}$$

$$\frac{\partial L}{\partial S_1} = \frac{\partial L}{\partial S_2} \frac{\partial S_2}{\partial \widehat{S}_2} \frac{\partial \widehat{S}_2}{\partial S_1}$$

Fully-connected Neural Network



$$\widehat{S}_i$$

$$S_i = \sigma(\overbrace{S_{i-1} W_i + b_i})$$

Suppose we know $\frac{\partial L}{\partial S_i}$

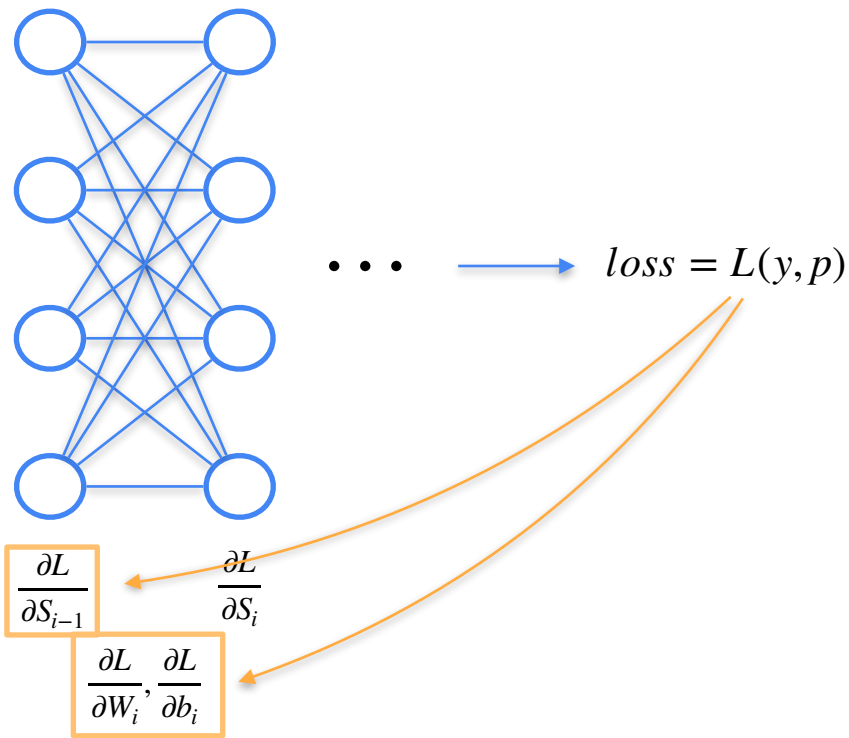
We also can get $\frac{\partial S_i}{\partial W_i}, \frac{\partial S_i}{\partial b_i}, \frac{\partial S_i}{\partial S_{i-1}}$

$$\frac{\partial S_i}{\partial W_i} = \frac{\partial S_i}{\partial \widehat{S}_i} \frac{\partial \widehat{S}_i}{\partial W_i}$$

$$\frac{\partial S_i}{\partial b_i} = \frac{\partial S_i}{\partial \widehat{S}_i} \frac{\partial \widehat{S}_i}{\partial b_i}$$

$$\frac{\partial S_i}{\partial S_{i-1}} = \frac{\partial S_i}{\partial \widehat{S}_i} \frac{\partial \widehat{S}_i}{\partial S_{i-1}}$$

Fully-connected Neural Network



$$\widehat{S}_i$$

$$S_i = \sigma(\overbrace{S_{i-1} W_i + b_i})$$

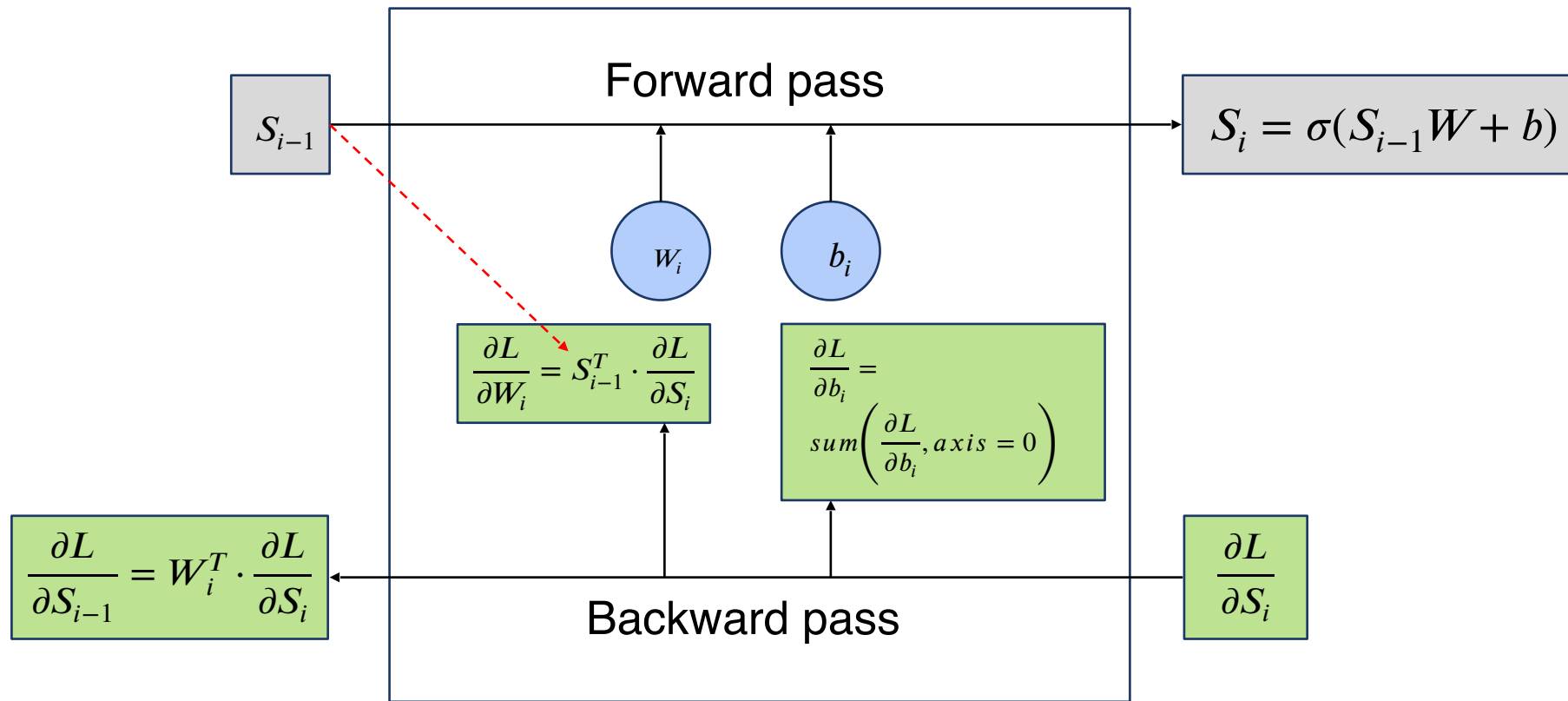
Then we can get:

$$\frac{\partial L}{\partial W_i} = \frac{\partial L}{\partial S_i} \frac{\partial S_i}{\partial \widehat{S}_i} \frac{\partial \widehat{S}_i}{\partial W_i}$$

$$\frac{\partial L}{\partial b_i} = \frac{\partial L}{\partial S_i} \frac{\partial S_i}{\partial \widehat{S}_i} \frac{\partial \widehat{S}_i}{\partial b_i}$$

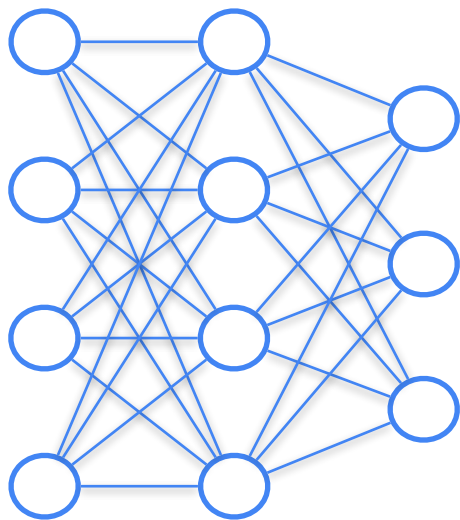
$$\frac{\partial L}{\partial S_{i-1}} = \frac{\partial L}{\partial S_i} \frac{\partial S_i}{\partial \widehat{S}_i} \frac{\partial \widehat{S}_i}{\partial S_{i-1}}$$

Fully-connected Layer



Activation functions

Why we need activation functions

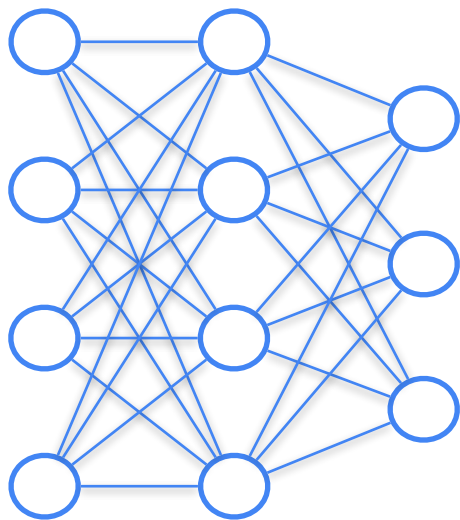


n_1 neurons n_2 neurons n_3 neurons

Layer formula:

$$\sigma(\sigma(XW_1 + b_1)W_2 + b_2)$$

Why we need activation functions



n_1 neurons n_2 neurons n_3 neurons

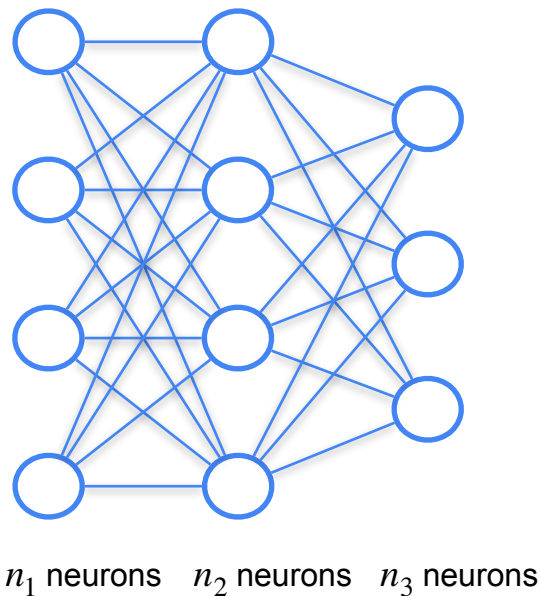
Layer formula:

$$\sigma(\sigma(XW_1 + b_1)W_2 + b_2)$$

What we get if there's no
activation function:

$$\sigma((XW_1 + b_1)W_2 + b_2)$$

Why we need activation functions



Layer formula:

$$\sigma(\sigma(XW_1 + b_1)W_2 + b_2)$$

What we get if there's no activation function:

$$\sigma((XW_1 + b_1)W_2 + b_2)$$

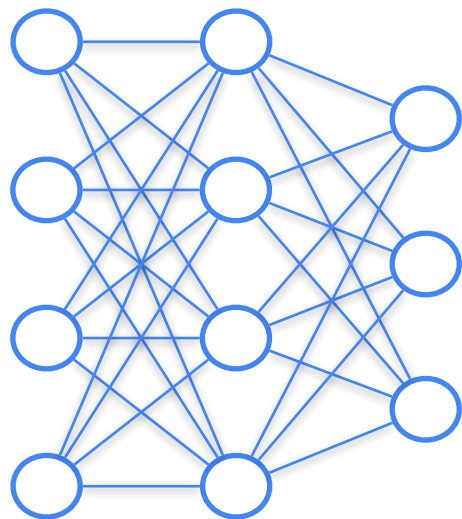
Now regroup variables:

$$\sigma(XW_1W_2 + (b_1W_2 + b_2))$$

In we change:
$$\begin{cases} W = W_1W_2 \\ b = b_1W_2 + b_2 \end{cases}$$

We will have: $\sigma(XW + b)$

Why we need activation functions



n_1 neurons n_2 neurons n_3 neurons

Layer formula:

$$\sigma(\sigma(XW_1 + b_1)W_2 + b_2)$$

What we get if there's no activation function:

$$\sigma(XW + b)$$

The fully-connected layer without activation function is called ***linear***, and activation function is called ***nonlinearity***

Activation functions

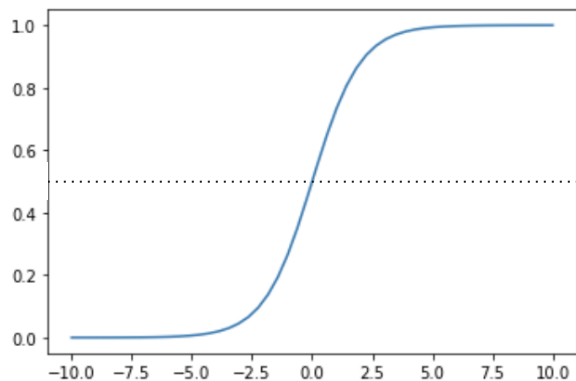
We'll consider three common types of activation functions:

- Sigmoid
- Tanh
- ReLU

There are other activation functions suitable for different kinds of tasks

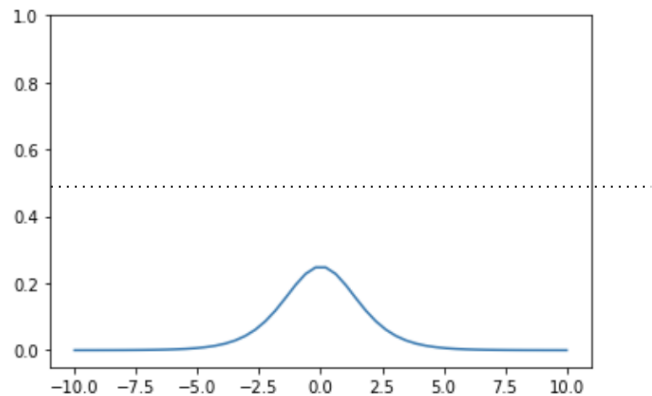
Sigmoid

Sigmoid function



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

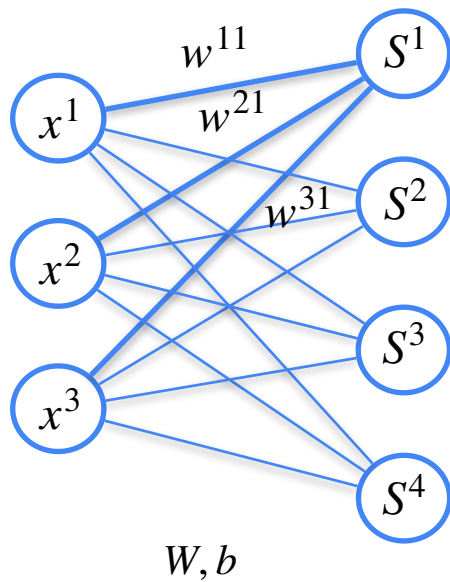
Derivative of Sigmoid



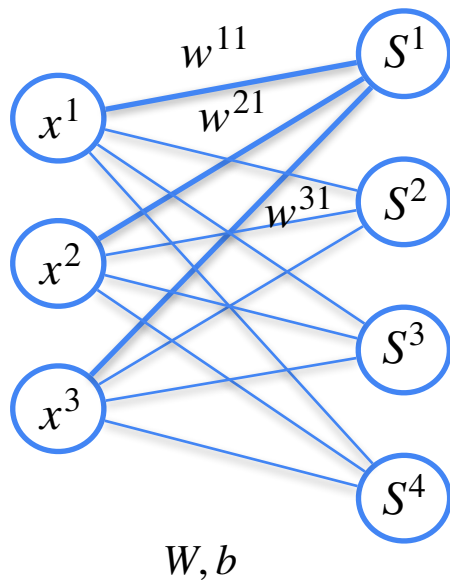
$$\sigma'(x) = \frac{e^{-x}}{(1 + e^{-x})^2}$$

Sigmoid

$$\hat{S} \\ S = \sigma(XW + b)$$



Sigmoid

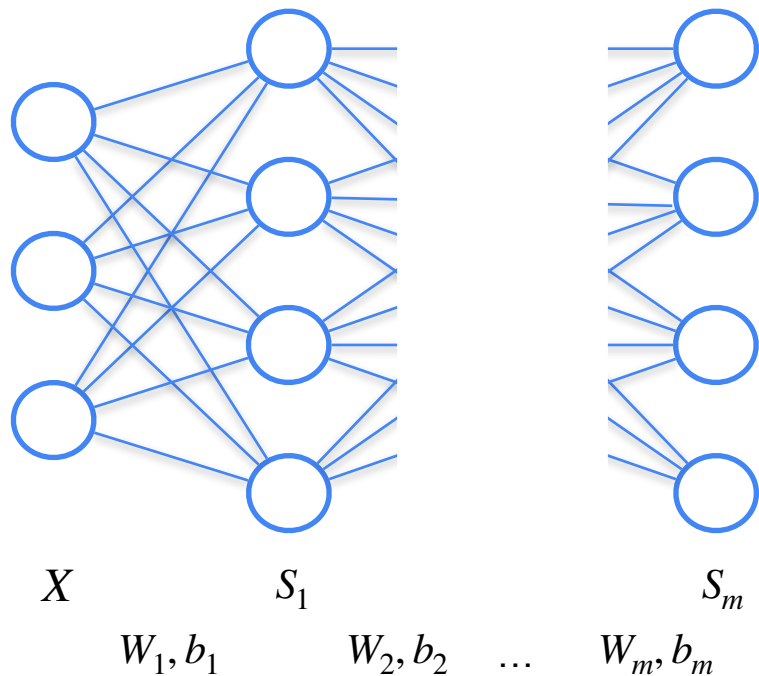


$$S = \sigma(\overbrace{XW + b}^{\hat{S}})$$

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial S} \boxed{\frac{\partial S}{\partial \hat{S}}} \frac{\partial \hat{S}}{\partial W}$$

Sigmoid derivative,
and it is close to zero

Sigmoid



$\hat{S}_i$

$S_i = \sigma(S_{i-1}W_i + b_i)$

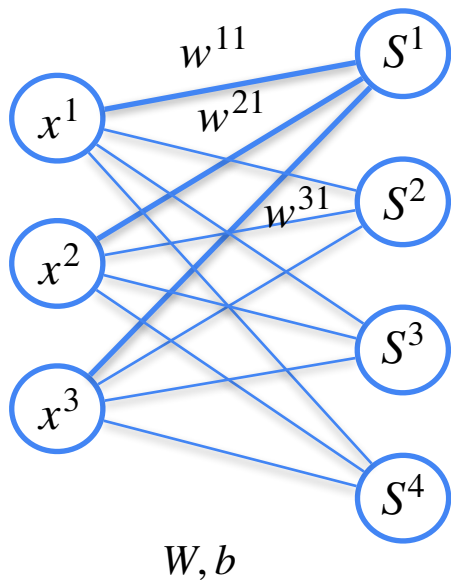
Sigmoid derivatives

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial S_m} \frac{\partial S_m}{\partial \hat{S}_m} \frac{\partial \hat{S}_m}{\partial S_{m-1}} \frac{\partial S_{m-1}}{\partial \hat{S}_{m-1}} \dots \frac{\partial S_1}{\partial \hat{S}_1} \frac{\partial \hat{S}_1}{\partial X}$$

If we have a deep network with sigmoid activation functions in each layer, the derivatives of first layers' parameters will get close to zero

This is called **vanishing gradients**

Sigmoid



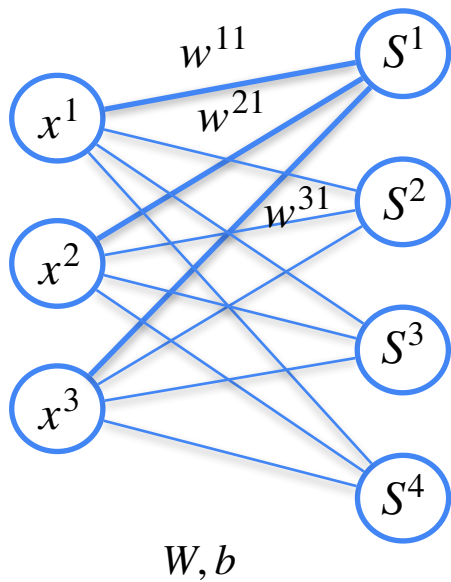
$$\hat{S}$$
$$S = \sigma(XW + b)$$

Formula for updating weights:

$$w^{i1} = w^{i1} - \alpha \frac{\partial L}{\partial w^{i1}} = w^{i1} - \alpha \frac{\partial L}{\partial S^1} \frac{\partial S^1}{\partial \hat{S}^1} \boxed{\frac{\partial \hat{S}^1}{\partial w^{i1}}}$$

||
 x^i

Sigmoid



$$\hat{S}$$
$$S = \sigma(XW + b)$$

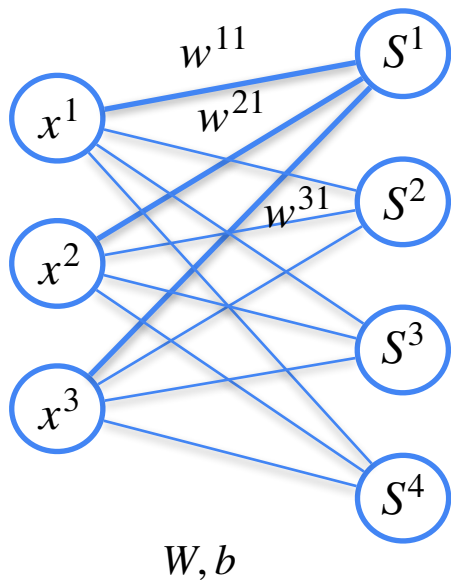
Formula for updating weights:

$$w^{i1} = w^{i1} - \alpha \frac{\partial L}{\partial w^{i1}} = w^{i1} - \alpha \frac{\partial L}{\partial S^1} \frac{\partial S^1}{\partial \hat{S}^1} x^i$$

Always positive as
sigmoid derivative

If previous layer has sigmoid
activation function, all the
 x^i will be positive

Sigmoid



$$\hat{S}$$
$$S = \sigma(XW + b)$$

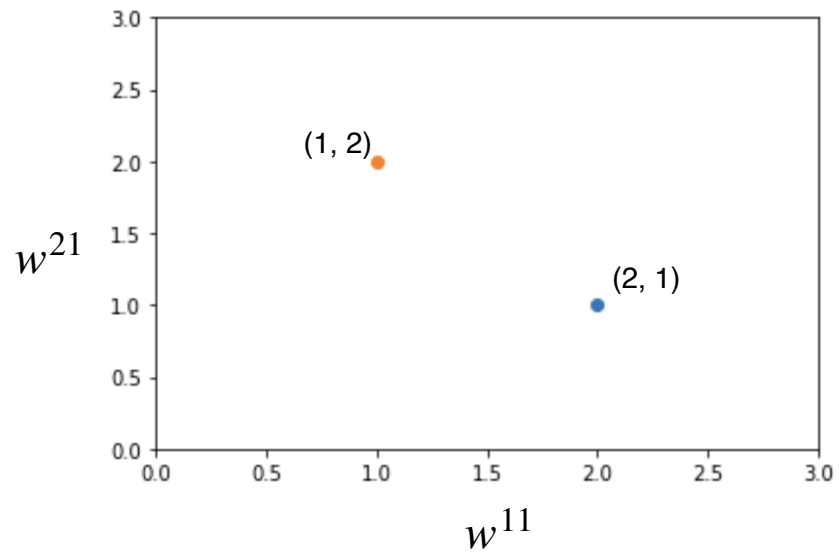
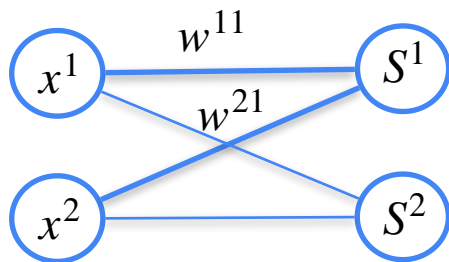
Formula for updating weights:

$$w^{i1} = w^{i1} - \alpha \frac{\partial L}{\partial w^{i1}} = w^{i1} - \alpha \frac{\partial L}{\partial S^1} \frac{\partial S^1}{\partial \hat{S}^1} x^i$$

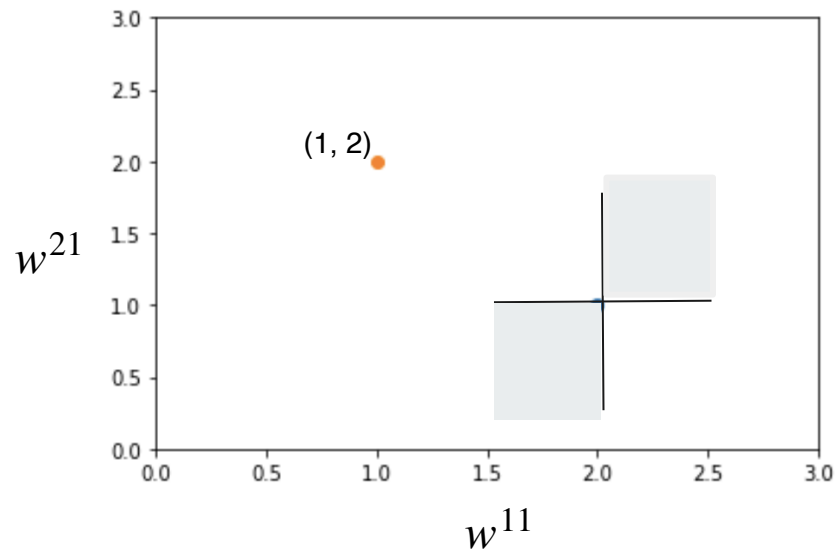
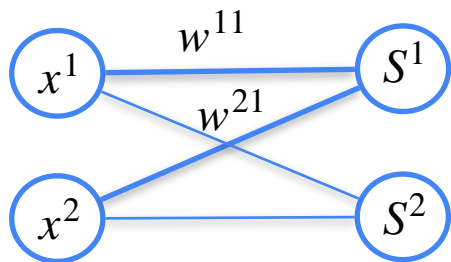
All the weights w^{i1} of this neuron will change by the value of one sign

Always positive

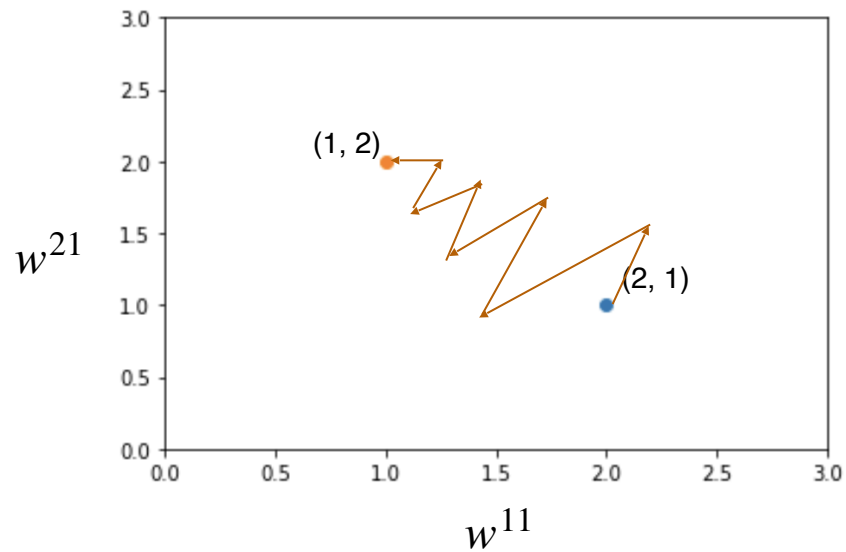
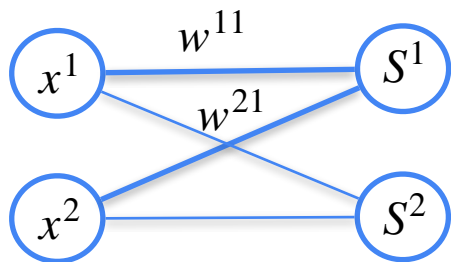
Sigmoid



Sigmoid



Sigmoid



Sigmoid

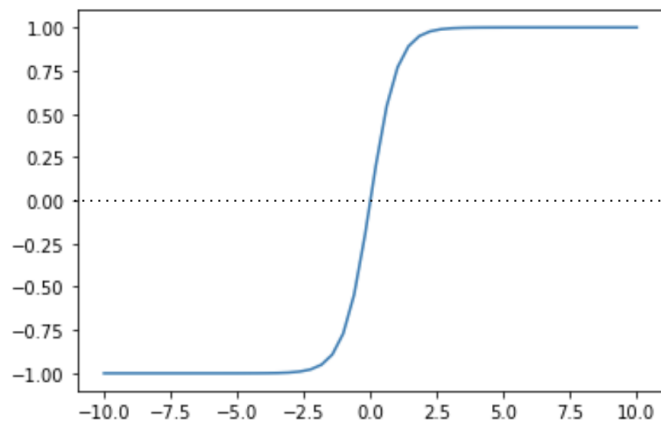
Limitations of Sigmoid activation function:

- Gradients are close to zero everywhere, leading to vanishing gradient
- Outputs of sigmoid are always positive, which makes training less effective

Note that in case of binary classification problem we still want to use Sigmoid as activation function of the last layer!

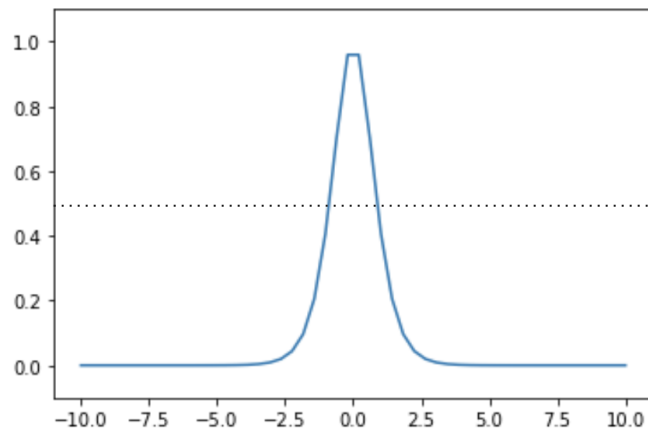
Tanh

Tanh function



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Derivative of Tanh

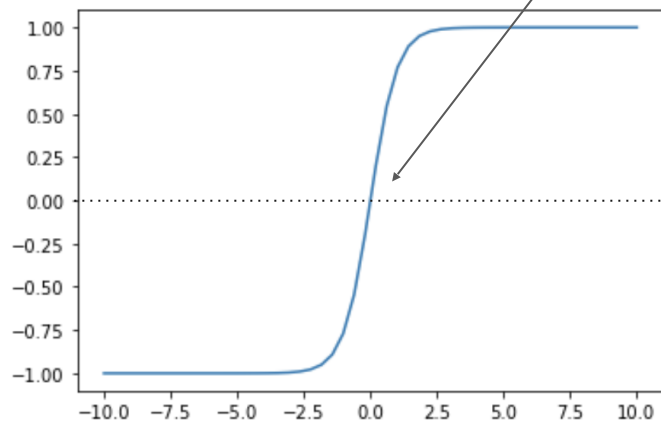


$$\tanh'(x) = 1 - \left(\frac{e^x - e^{-x}}{e^x + e^{-x}} \right)^2$$

Tanh

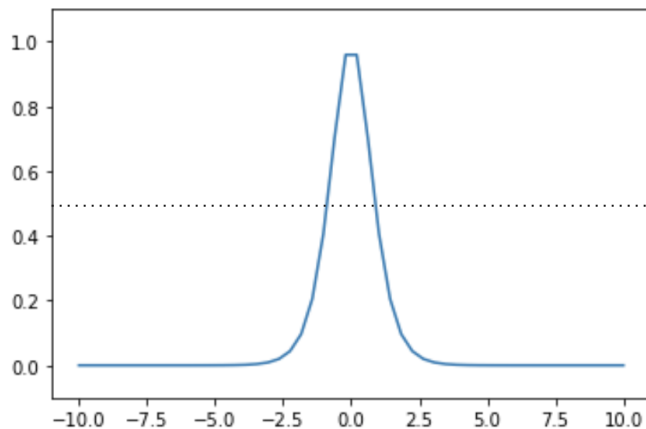
It is centered around
zero

Tanh function



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Derivative of Tanh

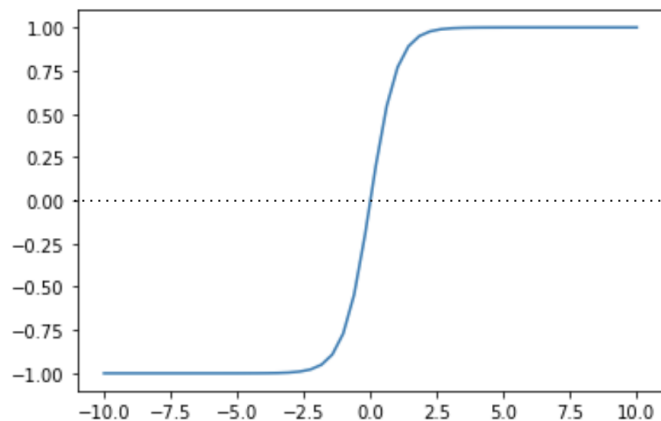


$$\tanh'(x) = 1 - \left(\frac{e^x - e^{-x}}{e^x + e^{-x}} \right)^2$$

Tanh

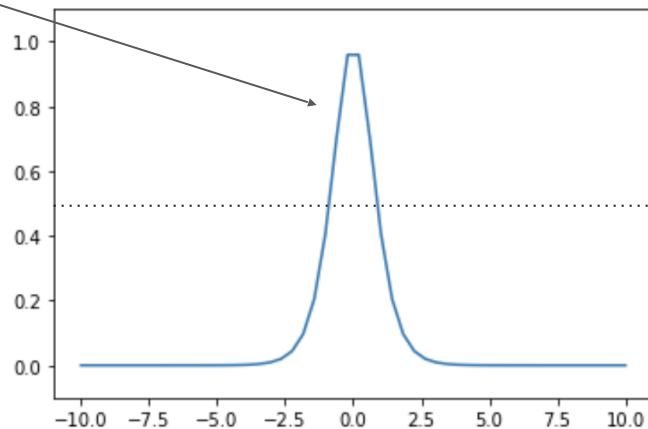
Derivative values
around zero are higher

Tanh function



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

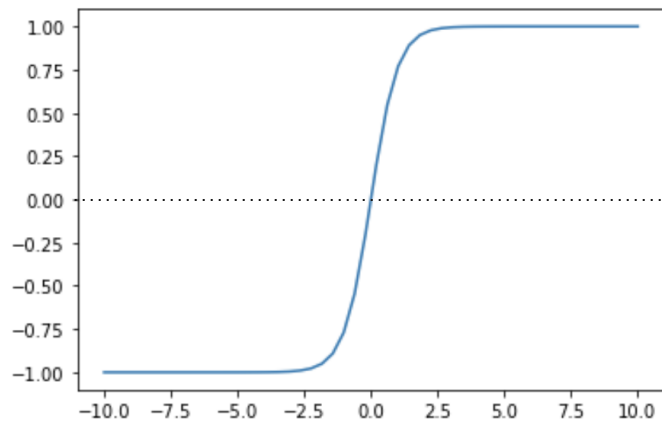
Derivative of Tanh



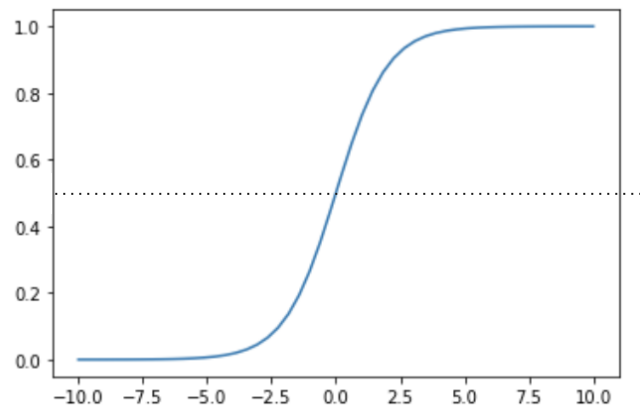
$$\tanh'(x) = 1 - \left(\frac{e^x - e^{-x}}{e^x + e^{-x}} \right)^2$$

Tanh

Tanh function



Sigmoid function



Tahn function expressed through Sigmoid: $\tanh(x) = 2\sigma(2x) - 1$

Tanh

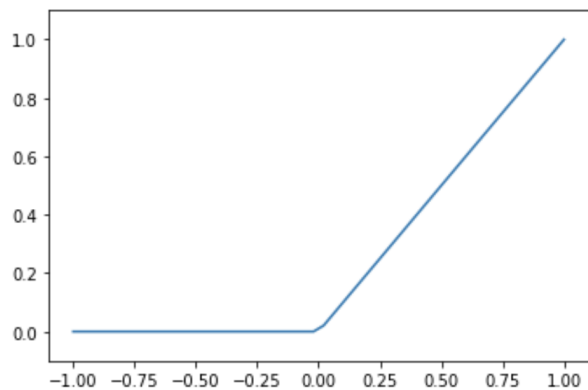
Advantages of Tang over Sigmoid:

- Tanh is centred around zero
- Gradients near zero are big

Despite this, gradients of Tang are still close to zero almost everywhere, which provokes vanishing gradient problem.

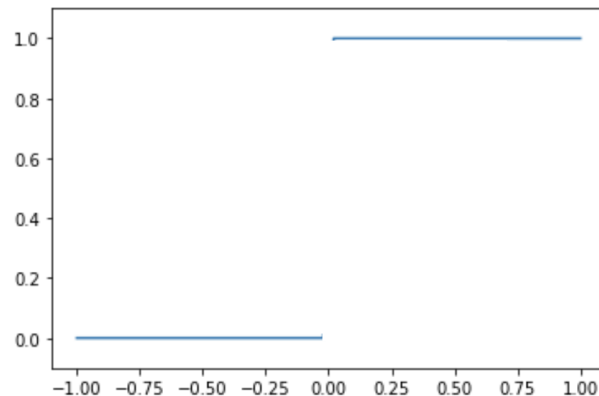
ReLU (Rectified Linear Unit)

ReLU function



$$ReLU(x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases}$$

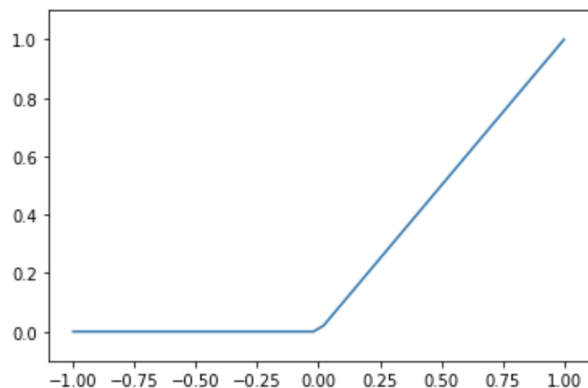
Derivative of ReLU



$$ReLU'(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases}$$

ReLU Problems

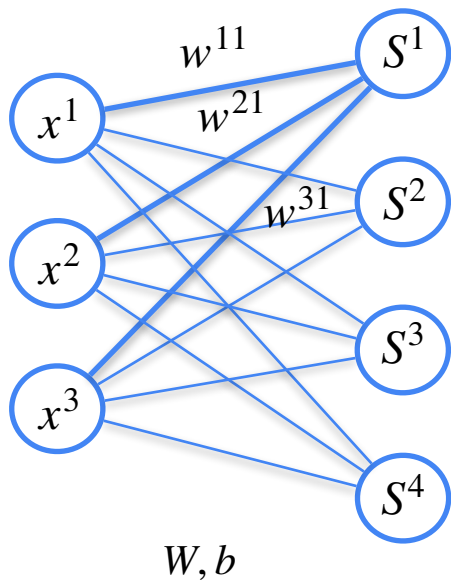
ReLU function



$$ReLU(x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases}$$

← Outputs of ReLU are always non-negative, so we have the same problem as in Sigmoid

ReLU



$$\hat{S}$$
$$S = \sigma(XW + b)$$

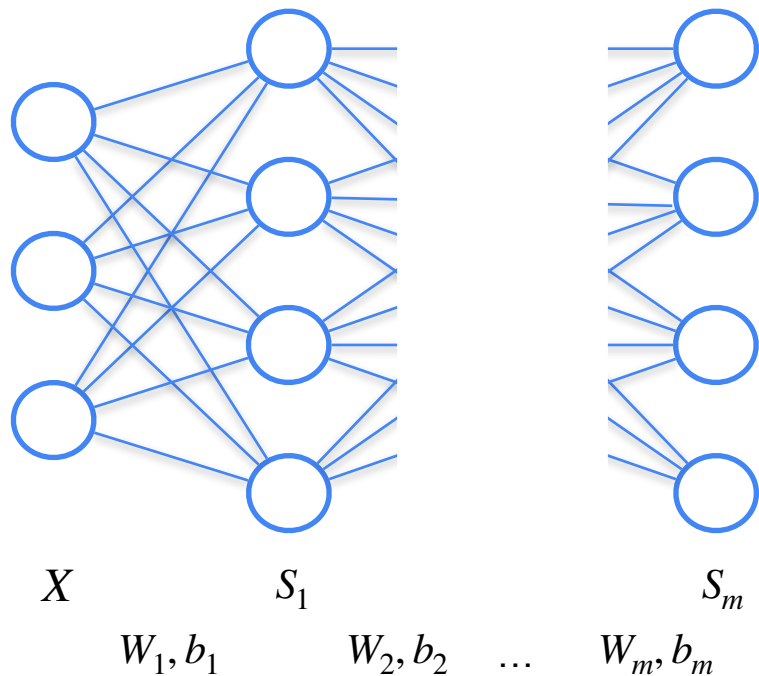
$$\frac{\partial L}{\partial w^{i1}} = \frac{\partial L}{\partial S} \frac{\partial S}{\partial \hat{S}} \frac{\partial \hat{S}}{\partial w^{i1}}$$

" x^i "

ReLU derivative, and it is often 1

If previous layer has ReLU activation function, x^i can be big

ReLU



ReLU derivatives

Output of previous layer

$$S_i = \sigma(\underbrace{\hat{S}_i}_{\text{ReLU derivatives}} W_i + b_i)$$

$$\frac{\partial L}{\partial W_i} = \frac{\partial L}{\partial S_m} \frac{\partial S_m}{\partial \hat{S}_m} \frac{\partial \hat{S}_m}{\partial S_{m-1}} \frac{\partial S_{m-1}}{\partial \hat{S}_{m-1}} \dots \frac{\partial S_{i+1}}{\partial \hat{S}_i} \frac{\partial \hat{S}_i}{\partial W_i}$$

Diagram illustrating the chain rule for the derivative of the loss L with respect to the weight W_i in a ReLU network. The derivative is expressed as a product of partial derivatives across layers, showing how the gradient flows back through the network. The diagram also shows the relationship between the input S_{i-1} and the output S_i of the previous layer.

If we have a deep network with ReLU activation functions in each layer, the derivatives of some layers' parameters can get big in amplitude. This will lead to big amplitude of weights

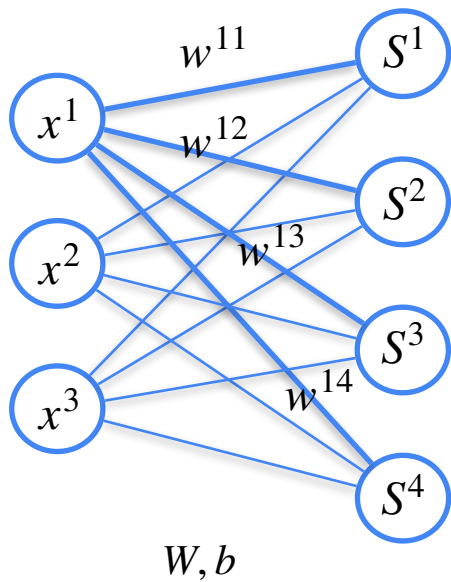
This is called **exploding gradients**

Controlling exploding gradients

The problem of exploding gradients can be controlled using the following:

- Clever initialisation of model's weights
- Optimal value of learning rate
- Normalisation of layers' outputs (we'll talk about it further in this lecture)
- Some other tricks

ReLU



$$\hat{S}$$
$$S = \sigma(XW + b)$$

Formula for updating weights:

$$w^{1i} = w^{1i} - \alpha \frac{\partial L}{\partial w^{1i}} = w^{1i} - \alpha \frac{\partial L}{\partial S^i} \frac{\partial S^i}{\partial \hat{S}^i} \underbrace{\frac{\partial \hat{S}^i}{\partial w^{1i}}}_{x^i}$$

It might happen that x^1 becomes zero for every possible input

Then weights w^{1i} will stop updating and the neuron becomes “dead”

ReLU

Advantages:

- Fast to compute (both forward and backward)
- Does not cause vanishing gradients so much
- It is close to the actual form of activation functions in biological neurons

Disadvantages:

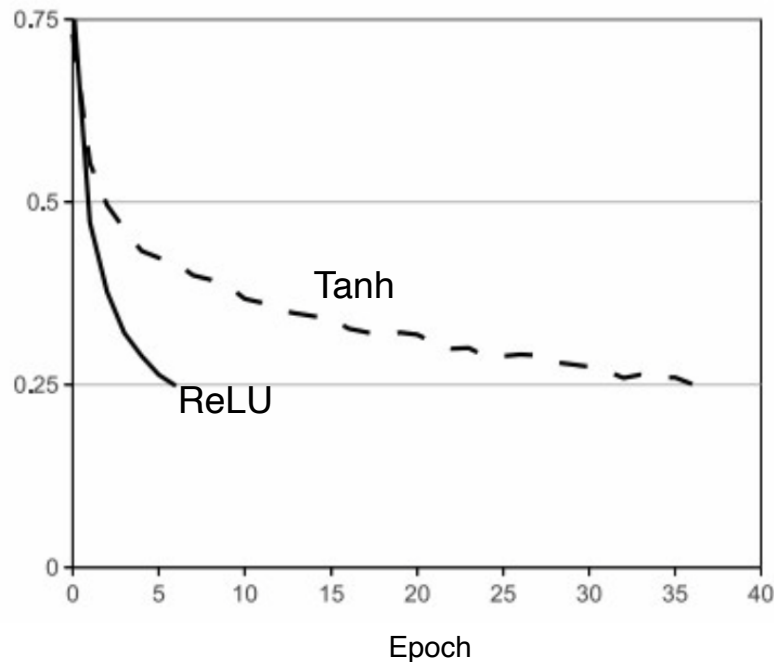
- Can cause exploding gradients
- Can cause “dead” neurons
- Outputs are always non-negative, which makes training less effective

ReLU

Despite all the disadvantages, ReLU is highly effective in practice.

It is one of the most widely used activation functions for hidden layers.

Loss function

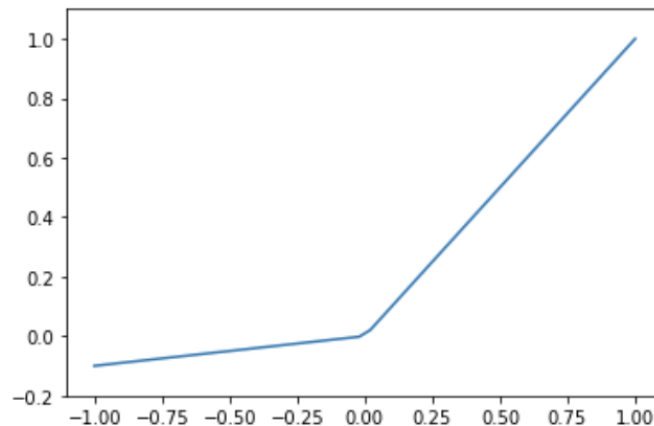


Leaky ReLU

Parameter α is not trainable

Usually α is really small, like 0.01

Leaky ReLU function



$$\text{Leaky ReLU}(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha x, & \text{otherwise} \end{cases}$$

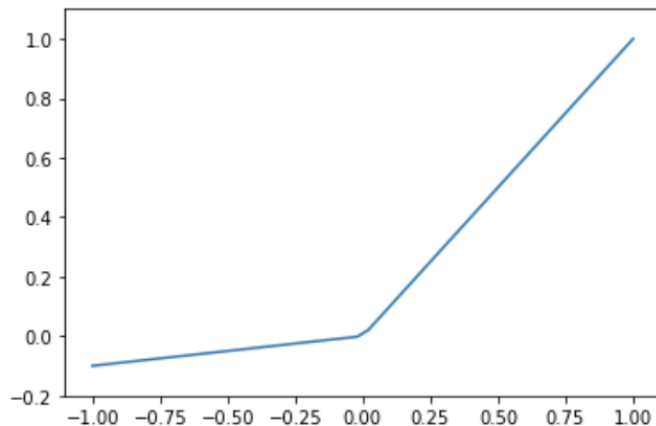
Leaky ReLU

Advantages over ReLU:

- no “dead” neurons problem
- outputs are not always non-negative

And it's still fast to compute

Leaky ReLU function

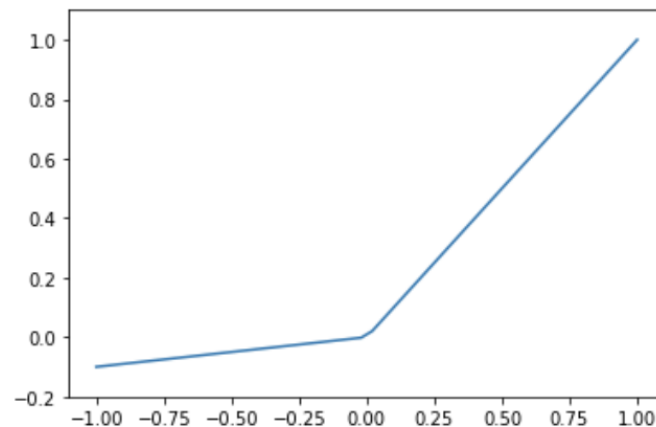


$$\text{Leaky ReLU}(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha x, & \text{otherwise} \end{cases}$$

Parametric ReLU

A version of Leaky ReLU, but with α being trainable

Leaky ReLU function



$$\text{Parametric ReLU}(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha x, & \text{otherwise} \end{cases}$$

Activation functions

- Using ReLU as activation function of hidden layers can be a default option
- Sigmoid is used as activation function of last layer in case of binary classification problem
- Sigmoid and Tanh are still used in some architectures (e.g. recurrent neural networks)
- There are many other different types of activation functions designed for different cases

Recap

In this class we have learned:

- Logistic Regression → Fully-Connected Neural Network
- Training of neural networks, idea of backpropagation
- Solving binary classification and regression tasks with neural networks
- Variants of activation functions
- Building neural network on PyTorch